

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA: CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN NHÓM
MÔN: MẠNG MÁY TÍNH
ĐỀ TÀI: THIẾT KẾ VÀ VẬN HÀNH ỨNG DỤNG HYBRID
FTP

Giảng viên hướng dẫn:	Nguyễn Thanh Quân
Lớp:	25C07
Học kỳ / Năm học:	III / 2025-2026
Thành viên 1 (25127446):	Nguyễn Minh Nhật
Thành viên 2 (25127320):	Huỳnh Nhật Hạ
Thành viên 3 (25127512):	Lê Hữu Anh Thư

Lời cảm ơn

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn **Nguyễn Thanh Quân** vì những góp ý chuyên môn trong quá trình thực hiện đồ án. Nhóm cũng ghi nhận sự hỗ trợ của các công cụ phát triển và trợ lý AI gồm Claude, ChatGPT, Codex, Antigravity, Gemini Pro, OpenCode và Copilot trong việc tra cứu, rà soát và hoàn thiện tài liệu.

Mọi quyết định thiết kế, nội dung kỹ thuật, kiểm thử và kết quả trình bày trong báo cáo đều được các thành viên kiểm tra và chịu trách nhiệm.

Mục lục

Lời cảm ơn	i
1 Application Scenario & Protocol Interaction	1
1.1 Khởi tạo và duy trì kênh TCP Control	1
1.2 Phiên UDP Data ở chế độ Passive	2
1.3 Phiên UDP Data ở chế độ Active	3
1.4 Lỗi và ngắt truyền giữa chừng	4
1.5 Nguồn đối chiếu trong dự án	4
2 Control Protocol & Packet Formats	4
2.1 TCP Control Packet Format	4
2.1.1 Cấu trúc lệnh từ client	5
2.1.2 Cấu trúc phản hồi từ server	5
2.1.3 Ý nghĩa các nhóm mã phản hồi	6
2.1.4 Các mã phản hồi đã cấu hình	6
2.1.5 Quan hệ giữa phản hồi TCP và dữ liệu UDP	7
2.2 UDP Custom Header Fields	7
2.2.1 Mô tả các trường	8
2.2.2 Wire format hoàn chỉnh	9
3 Kiến trúc và cơ chế truyền dữ liệu	10
3.1 Server Thread-Dispatch Logic và Active/Passive Mode	10
3.1.1 Kiến trúc xử lý đa luồng trên server	10
3.1.2 Chuyển đổi Active/Passive Mode	12
3.1.3 Tóm tắt kiến trúc tổng thể	15
3.2 RDT Sender/Receiver State Machines	15
3.2.1 FSM 1 — RDTWindowSender	16
3.2.2 FSM 2 — RDTWindowReceiver	18
4 Task Assignment Matrix — Phân công công việc	21

4.1	Phân công tổng quan theo giai đoạn	21
4.2	Ma trận phân công theo module/file	23
4.3	Biểu đồ Gantt phân công công việc	23
4.4	Thống kê đóng góp theo dòng code (ước tính)	25
5	Self-Assessment & Peer Evaluation	25
5.1	Tự đánh giá kết quả dự án	26
5.2	Tự đánh giá theo thành viên	27
5.3	Peer Evaluation	27
5.4	Hạn chế và kế hoạch cải tiến	28
6	GenAI Usage & Code Refinement Log	29
6.1	Phạm vi sử dụng GenAI	29
6.2	Lịch sử trao đổi theo thành viên	29
6.2.1	Dev 1 — Nguyễn Minh Nhật	29
6.2.2	Dev 2 — Huỳnh Nhật Hạ	29
6.2.3	Dev 3 — Lê Hữu Anh Thư	29
6.3	Nguyên tắc kiểm chứng và tinh chỉnh code	30
7	Application Demo Evidence	31
7.1	Tổng quan các ca kiểm thử	31
7.2	Chuẩn bị môi trường hai máy tính	31
7.2.1	Sơ đồ kết nối mạng	32
7.2.2	Kiểm tra kết nối mạng	32
7.2.3	Biên dịch dự án trên cả hai máy	32
7.2.4	Chuẩn bị dữ liệu test	32
7.3	Ca kiểm thử 1: Upload file thành công (STOR + RDT)	33
7.3.1	Mục đích	33
7.3.2	Các bước thực hiện	33
7.3.3	Bằng chứng thu được	34
7.4	Ca kiểm thử 2: Download file thành công (RETR + RDT)	35
7.4.1	Mục đích	35

7.4.2	Các bước thực hiện	35
7.4.3	Bảng chứng thu được	36
7.5	Ca kiểm thử 3: Kiểm tra SHA-256 toàn vẹn dữ liệu	37
7.5.1	Mục đích	37
7.5.2	Các bước thực hiện	37
7.5.3	Bảng chứng thu được	38
7.6	Ca kiểm thử 4: Danh sách phiên kết nối Client	39
7.6.1	Mục đích	39
7.6.2	Các bước thực hiện	39
7.7	Ca kiểm thử 5: Multiple concurrent clients	40
7.7.1	Mục đích	40
7.7.2	Các bước thực hiện	40
7.7.3	Bảng chứng thu được	42
7.8	Ca kiểm thử 6: Transfer file ASCII Mode (TYPE A)	43
7.8.1	Mục đích	43
7.8.2	Các bước thực hiện	43
7.8.3	Bảng chứng thu được	44
7.9	Ca kiểm thử 7: Transfer file Binary Mode (TYPE I)	45
7.9.1	Mục đích	45
7.9.2	Các bước thực hiện	45
7.9.3	Bảng chứng thu được	46
7.10	Tóm tắt bằng chứng	47
7.10.1	Bảng kiểm tra hoàn thành	47
7.10.2	Lưu giữ evidence files	47
7.11	Dọn dẹp sau khi test	48

1 Application Scenario & Protocol Interaction

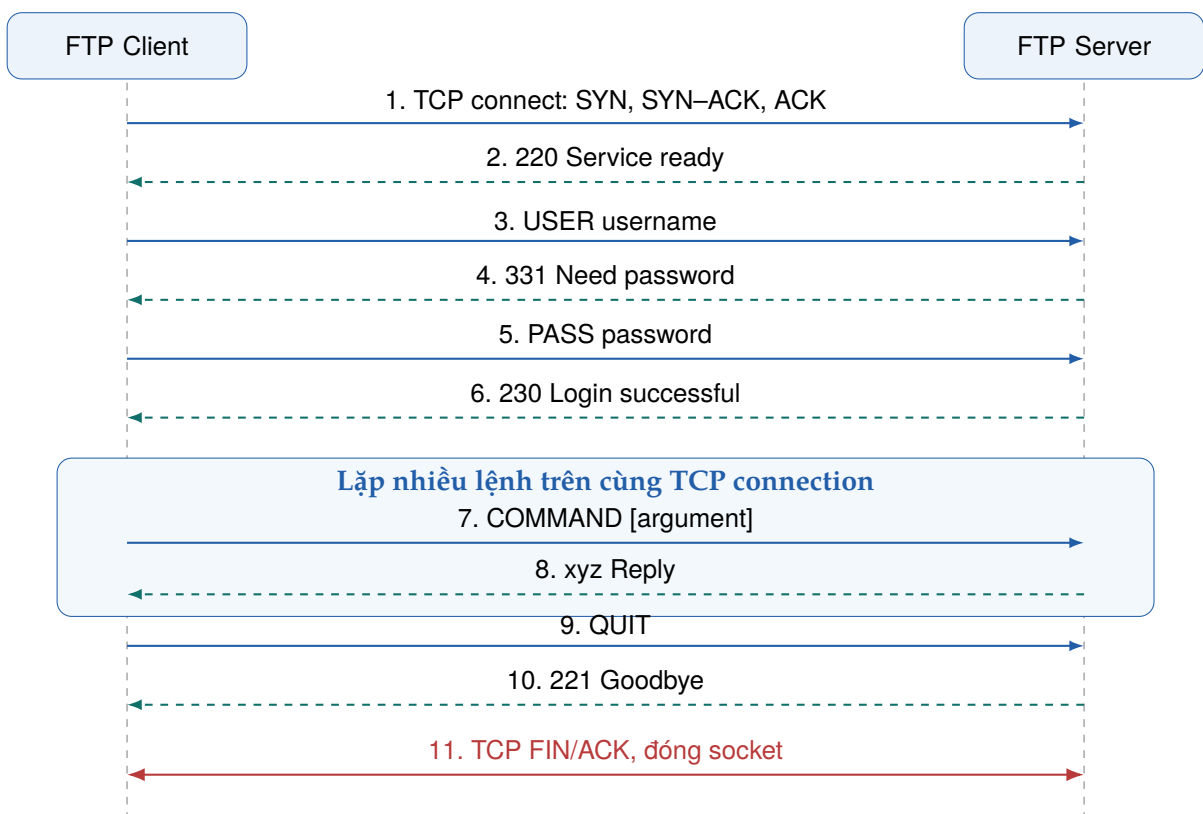
Hệ thống Hybrid FTP duy trì một kết nối **TCP Control** lâu dài để trao đổi lệnh và phản hồi. Với mỗi thao tác LIST, NLST, RETR, STOR, STOU hoặc APPE, hệ thống tạo một phiên **UDP-RDT** dùng một lần để vận chuyển dữ liệu.

Cách hiểu “kết nối UDP”

UDP là giao thức không kết nối. Trong báo cáo, cụm từ “mở/ngắt kết nối UDP” chỉ quá trình tạo socket, xác định cặp IP/port, chạy phiên RDT và đóng socket. UDP không có bắt tay hoặc thủ tục đóng kết nối ở tầng vận chuyển như TCP.

1.1 Khởi tạo và duy trì kênh TCP Control

Server khởi tạo socket, bind cổng 2121 và gọi listen(). Sau khi accept(), mỗi client được gán cho một thread và một SessionState riêng. Kết nối TCP tiếp tục tồn tại qua nhiều lệnh FTP cho đến khi nhận QUIT, EOF hoặc lỗi mạng.



Hình 1: Vòng đời của kênh TCP Control.

Bảng 1: Các cách kết thúc session không đi theo luồng QUIT thông thường.

Trường hợp	Trình tự xử lý
EOF hoặc lỗi mạng	recv() trả 0/lỗi; server đóng TCP socket và kết thúc session.
Client nhận SIGINT/SIGTERM	Client gửi ABOR; server trả 426 rồi 226; client gửi QUIT, nhận 221 và đóng TCP.
Client đóng đột ngột	Vòng đọc lệnh kết thúc; SessionState bị hủy và passive UDP socket còn lại được giải phóng bởi unique_ptr.

SessionState lưu trạng thái đăng nhập, thư mục hiện hành, kiểu truyền, chế độ ACTIVE/PASSIVE và passive UDP socket. QUIT và NOOP được chấp nhận trước khi đăng nhập; các lệnh còn lại yêu cầu trạng thái LoggedIn.

1.2 Phiên UDP Data ở chế độ Passive

Client hiện tại tự động chọn Passive Mode cho các lệnh truyền dữ liệu. Server bind một UDP port động và trả địa chỉ qua mã 227. TCP chỉ điều phối; DATA, ACK, FIN và FINACK luôn đi trên UDP.

Bảng 2: Passive Mode khi server gửi dữ liệu: RETR, LIST và NLST.

Bước	Trao đổi	Ý nghĩa
1	TCP : Client → Server: PASV	Server mở UDP port động.
2	TCP : Server → Client: 227	Client nhận server IP và UDP port.
3	UDP : Client → Server: knock 15 byte	Server học endpoint IP:port của client.
4	TCP : RETR/LIST/NLST; phản hồi 150	Bắt đầu phiên data transfer.
5	UDP : DATA ↔ ACK tích lũy	Server là RDT sender; client quảng bá receive window.
6	UDP : FIN(<i>N</i>) ↔ FIN+ACK(<i>N</i>)	Kết thúc tin cậy phiên RDT.
7	TCP : Server → Client: 226	Đóng UDP socket; TCP Control vẫn mở.

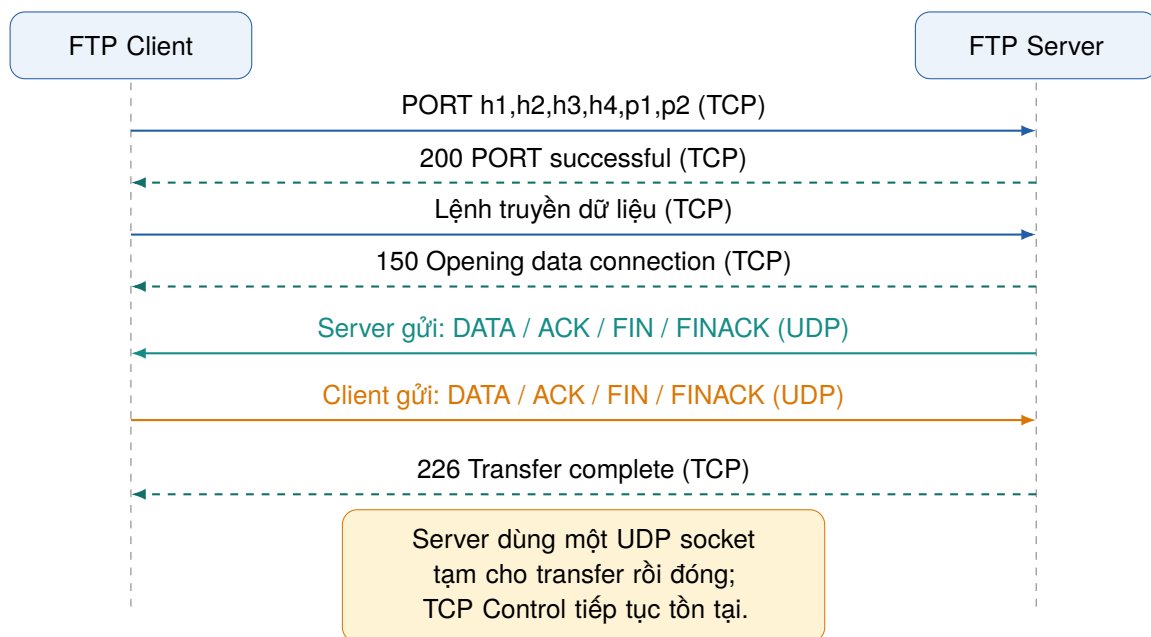
Bảng 3: Passive Mode khi client gửi dữ liệu: STOR, STOU và APPE.

Bước	Trao đổi	Ý nghĩa
1	TCP : PASV → 227	Server tạo passive socket và công bố port.
2	TCP : STOR/STOU/APPE → 150 hoặc 125	Server chuẩn bị nhận và client mở UDP socket.
3	UDP : DATA(0 đến $N - 1$) ↔ ACK	DATA đầu tiên đồng thời cung cấp endpoint client, nên không cần knock riêng.
4	UDP : FIN(N) ↔ FIN+ACK(N)	Receiver xác nhận kết thúc luồng.
5	Server ghi/append file và tính SHA-256	Chỉ trả thành công sau khi xử lý file hoàn tất.
6	TCP : Server → Client: 226	Hai phía đóng UDP socket; TCP Control vẫn mở.

Nếu timeout, mất gói hoặc checksum sai, receiver gửi lại ACK tích lũy cũ. Sender đặt lại congestion window, đưa nextSeq về base và truyền lại theo Go-Back-N. Knock chỉ cần cho chiều server gửi vì server chưa biết UDP endpoint của client; trong chiều upload, DATA đầu tiên thực hiện vai trò này.

1.3 Phiên UDP Data ở chế độ Active

Với lệnh PORT h1, h2, h3, h4, p1, p2, server lưu địa chỉ client và tính port theo $p_1 \times 256 + p_2$. Client phải mở và bind UDP port trước khi gửi PORT.



Hình 2: Trình tự tham chiếu của Active Mode.

1.4 Lỗi và ngắt truyền giữa chừng

Mã	Điều kiện và xử lý
425	Chưa có PORT/PASV, không mở được socket hoặc Passive Mode không nhận được knock.
426	RDT vượt quá số vòng retransmit, mất FINACK, ABOR hoặc transfer thất bại.
551	Receiver đã nhận dữ liệu nhưng server không ghi được file đích.
426, 226	ABOR đóng passive socket nếu có, đặt dataMode=NONE và phản hồi best-effort. Hiện tại chưa có cờ hủy đồng bộ để dừng tức thời RDT đang chạy trong cùng thread.

Trình tự tham chiếu ngắn

TCP connect → 220 → USER/331 → PASS/230
 → PASV/227 hoặc PORT/200 → lệnh truyền/125 hoặc 150
 → UDP DATA ↔ ACK → FIN ↔ FINACK
 → TCP 226 → lệnh tiếp theo → QUIT/221.

1.5 Nguồn đối chiếu trong dự án

- src/server/ServerManager.cpp: tạo, bind, listen, accept và dispatch thread;
- src/server/Session.cpp: control loop, command dispatch, ACTIVE/PASSIVE và reply code;
- src/client/main_client.cpp: kết nối TCP, framing CRLF và đọc reply;
- src/client/ClientCLI.cpp: PASV, UDP socket, knock và orchestration transfer;
- src/rdt/CustomUDPHeader.h: header và cờ RDT;
- src/rdt/SlidingWindow.cpp: Go-Back-N, ACK tích lũy, flow/congestion control và FIN/FINACK.

2 Control Protocol & Packet Formats

2.1 TCP Control Packet Format

TCP cung cấp một byte stream, vì vậy Hybrid FTP không định nghĩa header control riêng. Đơn vị bản tin ở tầng ứng dụng là một dòng kết thúc bằng CRLF (\r\n).

2.1.1 Cấu trúc lệnh từ client

Application framing

<COMMAND>[SP <ARGUMENT>] CRLF

Trường	Kích thước	Quy tắc
COMMAND	Biến đổi	Tên lệnh ASCII; parser chuyển sang chữ hoa, ví dụ USER, PASV, RETR.
SP	1+ byte	Một hoặc nhiều dấu cách/tab; không xuất hiện nếu lệnh không có đối số.
ARGUMENT	Biến đổi	Phần còn lại sau tên lệnh; khoảng trắng hai đầu bị loại bỏ.
CRLF	2 byte	0x0D0x0A; server đọc đến LF và bỏ CR.

Listing 1: Ví dụ các dòng lệnh TCP Control.

```

1 USER alice\r\n
2 TYPE I\r\n
3 RETR reports/data.bin\r\n
4 PORT 192,168,1,10,195,80\r\n
5 QUIT\r\n

```

Client thêm CRLF và dùng `sendAll()` để xử lý trường hợp `send()` chỉ gửi một phần buffer. Server dùng `char line[512]`, tương ứng tối đa 511 ký tự hữu ích; lệnh dài hơn giới hạn có thể bị tách và không nên sử dụng.

2.1.2 Cấu trúc phản hồi từ server

Phản hồi một dòng

<CODE> SP <MESSAGE> CRLF

Phản hồi nhiều dòng

<CODE>-<FIRST-LINE> CRLF
 <INTERMEDIATE-LINE> CRLF
 <CODE> SP <LAST-LINE> CRLF

CODE luôn gồm ba chữ số. Nếu ký tự thứ tư là dấu gạch ngang, client tiếp tục đọc đến dòng bắt đầu bằng cùng mã và một dấu cách. HELP và STAT có thể tạo phản hồi nhiều dòng.

Listing 2: Ví dụ phản hồi TCP Control.

```

1 220 Hybrid FTP service ready.\r\n
2 331 Username OK, need password.\r\n
3 227 Entering Passive Mode (127,0,0,1,195,80).\r\n
4 150 Opening data connection for file download.\r\n
5 226 Transfer complete.\r\n

```

2.1.3 Ý nghĩa các nhóm mã phản hồi

Nhóm	Ý nghĩa giao thức	Hành động phía client
1xx	Phản hồi sơ bộ tích cực; thao tác đã bắt đầu nhưng chưa hoàn tất.	Tiến hành data transfer và chờ phản hồi cuối.
2xx	Hoàn tất tích cực.	Có thể gửi lệnh tiếp theo.
3xx	Tích cực trung gian; cần thêm thông tin hoặc lệnh.	Gửi bước tiếp theo như PASS hoặc RNTD.
4xx	Lỗi tạm thời; có thể thử lại.	Sửa trạng thái data channel hoặc thử lại transfer.
5xx	Lỗi cú pháp, trạng thái, quyền hoặc thao tác vĩnh viễn.	Sửa lệnh, đăng nhập hoặc đường dẫn.

2.1.4 Các mã phản hồi đã cấu hình

Bảng 4: Reply code được sử dụng trong Hybrid FTP.

Mã	Bản tin/biến thể	Khi phát sinh
125	FILE: <uniqueName>	STOU chọn tên duy nhất và data transfer có thể bắt đầu.
150	Opening data connection for ...	Bắt đầu truyền file hoặc danh sách thư mục.
200	NOOP thành công; đặt TYPE/MODE; PORT thành công	Lệnh hoàn tất bình thường.
211	Server status / End of status	STAT không có đối số, dạng nhiều dòng.
212	Trạng thái đường dẫn	STAT với đối tượng là thư mục.
213	File status, SIZE, MDTM hoặc SHA-256	STAT file, SIZE, MDTM hoặc HASH.
214	Danh sách lệnh hoặc Syntax: ...	HELP hoặc HELP với một command.
220	Hybrid FTP service ready	Ngay sau khi accept TCP connection.
221	Goodbye	QUIT hợp lệ; session chuẩn bị đóng TCP.
226	Transfer/directory complete; abort successful	RDT hoàn tất hoặc ABOR đã xử lý best-effort.
227	Entering Passive Mode (...)	PASV mở thành công UDP socket phía server.
230	Login successful	PASS sau USER; hiện tại chưa kiểm tra mật khẩu thực tế.
250	File/directory action successful	CWD, CDUP, RMD, DELE hoặc RNTD thành công.
257	Đường dẫn hiện hành hoặc directory created	PWD hoặc MKD thành công.
331	Username OK, need password	USER có đối số.
350	Requested file action pending RNTD	RNFR hợp lệ; chờ RNTD.

Còn tiếp ở trang sau

Mã	Bản tin/biến thể	Khi phát sinh
425	Không có/không mở được data connection	Chưa PORT/PASV, bind lỗi hoặc không học được endpoint.
426	Connection closed; transfer aborted	RDT thất bại hoặc ABOR.
500	Invalid command	Dòng rỗng hoặc không parse được.
501	Thiếu/sai đối số hoặc sai định dạng	PORT, TYPE, MODE hay command parameter không hợp lệ.
502	Command not implemented / unknown command	Lệnh chưa hiện thực hoặc HELP hỏi lệnh không tồn tại.
503	USER first / RNFR before RNT0	Sai thứ tự lệnh.
530	Not logged in	Dùng lệnh được bảo vệ trước khi đăng nhập.
550	Not found, denied, outside root hoặc file action failed	Lỗi đường dẫn, quyền, filesystem hoặc tính hash.
551	Failed to write file	Đã nhận upload nhưng không ghi được file đích.

Các mã đang dùng gồm: 125 và 150 thuộc 1xx; 200, 211–214, 220, 221, 226, 227, 230, 250, 257 thuộc 2xx; 331 và 350 thuộc 3xx; 425 và 426 thuộc 4xx; 500–503, 530, 550 và 551 thuộc 5xx.

2.1.5 Quan hệ giữa phản hồi TCP và dữ liệu UDP

Một transfer thành công có hai phản hồi trên TCP: **125/150** báo server đã sẵn sàng để bắt đầu RDT, còn **226** chỉ được gửi sau FIN/FINACK và, với upload, sau khi ghi file thành công. Client phải đọc cả hai phản hồi. Dữ liệu file không đi trên TCP Control; ngược lại, ACK/FIN/FINACK trên UDP không thay thế các reply code FTP.

2.2 UDP Custom Header Fields

Phụ trách: Dev 2 — Reliable UDP và thuật toán RDT

Tổng quan

Dự án Hybrid FTP sử dụng một kênh dữ liệu UDP tùy chỉnh thay cho luồng TCP thông thường nhằm tăng thông lượng khi truyền tệp lớn. Vì UDP không bảo đảm thứ tự, xác nhận hay truyền lại, nhóm bổ sung một header dài **15 byte** trước mỗi payload, gọi là CustomUDPHeader.

Header được khai báo bằng `#pragma pack(push, 1)` để loại bỏ padding và bảo đảm kích thước trong bộ nhớ trùng với wire format. Tất cả trường nhiều byte được serialize theo **Network Byte Order (Big-Endian)** bằng `htonl/htons`; phía nhận chuyển ngược bằng `ntohl/ntohs`.

Bảng 5: Bố cục byte của CustomUDPHeader.

Byte offset	Kích thước	Tên trường	Kiểu C++
0–3	4 byte	seqNum	uint32_t
4–7	4 byte	ackNum	uint32_t
8–9	2 byte	payloadLen	uint16_t
10–11	2 byte	windowSize	uint16_t
12–13	2 byte	checksum	uint16_t
14	1 byte	flags	uint8_t

2.2.1 Mô tả các trường

seqNum — Sequence Number (4 byte). Số thứ tự của segment trong luồng truyền; đơn vị là segment, bắt đầu từ 0. Sender tăng dần trường này cho mỗi gói DATA. Receiver so sánh với expectedSeq_ để phát hiện gói lệch thứ tự theo Go-Back-N. Gói FIN sử dụng seqNum= N , trong đó N là tổng số segment đã gửi.

ackNum — Acknowledgment Number (4 byte). ACK tích lũy xác nhận rằng receiver đã nhận đúng thứ tự toàn bộ segment từ 0 đến ackNum; quan hệ với trạng thái nhận là ackNum=expectedSeq_ - 1. Trong DATA, trường này bằng 0; trong FINACK, ackNum bằng số thứ tự của FIN.

payloadLen — Payload Length (2 byte). Số byte dữ liệu đi sau header trong cùng datagram. Kiểu uint16_t cho phép giá trị tối đa 65,535 byte, nhưng cấu hình thực tế dùng MSS mặc định 1,024 byte. ACK, FIN và FINACK không mang dữ liệu nên có payloadLen=0.

windowSize — Window Size (2 byte). Trong ACK, receiver quảng bá advertisedWindow_ để sender giới hạn số gói đang bay. Trong DATA, sender ghi cửa sổ hiệu dụng min(cwnd, rcvWindow_, maxWindowSegments_). Khi cửa sổ nhận bằng 0, sender vẫn cho phép một segment thăm dò để tránh deadlock kiểu zero-window.

checksum — Internet Checksum (2 byte). Checksum được tính trên toàn bộ *header + payload*. Quá trình tính gồm:

1. đặt trường checksum trong bản sao header bằng 0;
2. serialize header rồi nối với payload;
3. cộng các word 16-bit và gộp carry về 16 bit;
4. lấy bù một: $\sim sum \ \& \ 0xFFFF$.

Nếu verifyChecksum() thất bại, receiver bỏ gói và gửi lại ACK tích lũy gần nhất.

flags — Control Flags (1 byte). Năm bit thấp mang các cờ điều khiển; ba bit cao được dự phòng.

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	NAK	FIN	DATA	ACK	SYN

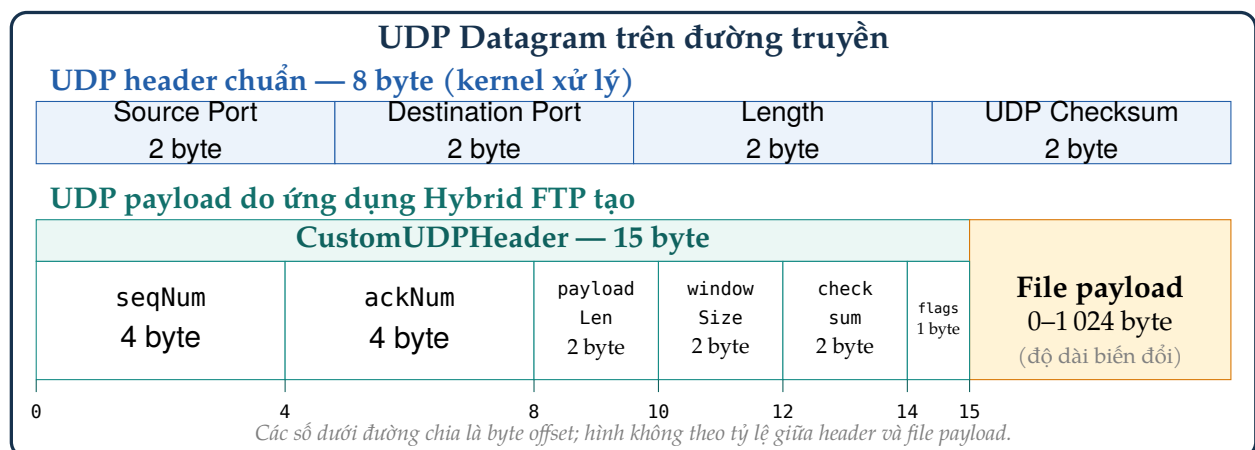
Hình 3: Bố cục bit của trường flags.

Bảng 6: Các cờ điều khiển.

Cờ	Bit	Hex	Ý nghĩa
FLAG_SYN	0	0x01	Thiết lập kết nối; dự phòng, hiện chưa sử dụng.
FLAG_ACK	1	0x02	Gói xác nhận dữ liệu.
FLAG_DATA	2	0x04	Gói mang payload của tệp.
FLAG_FIN	3	0x08	Kết thúc luồng truyền.
FLAG_NAK	4	0x10	Xác nhận âm; dành cho mở rộng.

Các tổ hợp đang dùng gồm DATA (FLAG_DATA), ACK (FLAG_ACK), FIN (FLAG_FIN) và FINACK (FLAG_FIN|FLAG_ACK). Việc bật, tắt và kiểm tra cờ được đóng gói trong setFlag, clearFlag và hasFlag.

2.2.2 Wire format hoàn chỉnh



Hình 4: Định dạng đầy đủ của datagram UDP dùng trong Hybrid FTP.

Bảng 7: So sánh UDP thuần và UDP có custom header.

Tiêu chí	UDP thuần	UDP + CustomUDPHeader
Đảm bảo thứ tự	Không	Có, nhờ seqNum và expectedSeq_.
Phát hiện lỗi	Checksum UDP cơ bản	Internet checksum trên toàn bộ header và dữ liệu.
Kiểm soát luồng	Không	Cửa sổ nhận được quảng bá trong ACK.
Kiểm soát tắc nghẽn	Không	Slow Start và AIMD tại sender.
Kết thúc luồng	Không có handshake riêng	FIN/FINACK xác nhận rõ ràng.

3 Kiến trúc và cơ chế truyền dữ liệu

3.1 Server Thread-Dispatch Logic và Active/Passive Mode

3.1.1 Kiến trúc xử lý đa luồng trên server

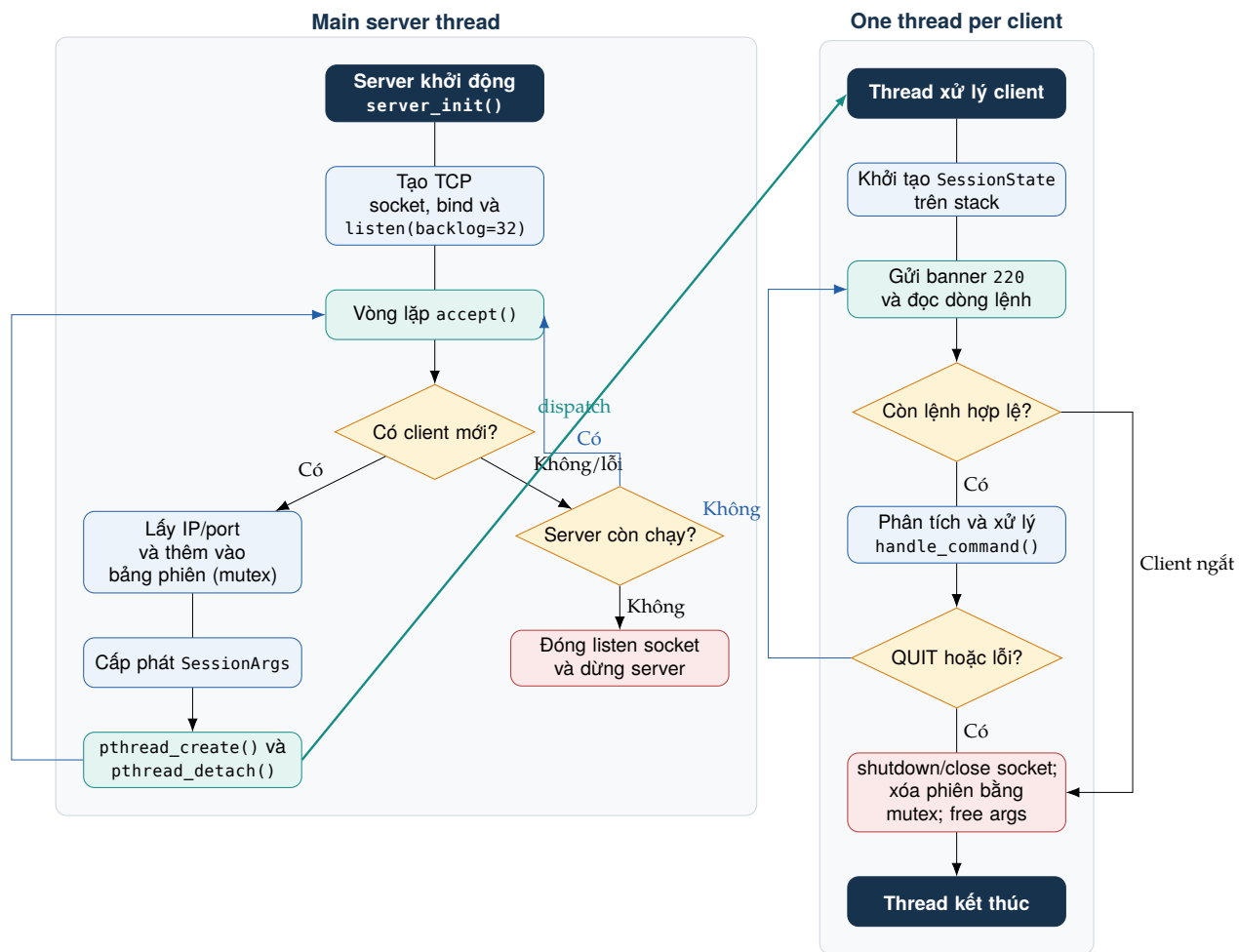
Hybrid FTP Server sử dụng mô hình **Thread-per-Client** trên POSIX Threads. Mỗi kết nối TCP được `accept()` sẽ tạo một thread chạy `handle_client_thread()`; thread được detach ngay để vòng lặp chính quay lại phục vụ kết nối tiếp theo.

Listing 3: Cấu trúc quản lý server và bảng phiên.

```

1 #define MAX_CLIENTS 100
2
3 struct ClientRecord {
4     int socketFd;
5     char ip[46];
6     int port;
7     int active;
8 };
9
10 struct ServerState {
11     int listenFd;
12     int port;
13     int running;
14     pthread_mutex_t clientsLock;
15     ClientRecord clients[MAX_CLIENTS];
16 };

```



Hình 5: Luồng dispatch và vòng đời của một client thread.

Tài nguyên	Cơ chế bảo vệ	Giải thích
ServerState::clients[]	pthread_mutex_tclients Lock	Lock khi thêm hoặc xóa client khỏi bảng phiên.
SessionState	Biến cục bộ trên stack	Mỗi thread có một bản sao riêng, không cần đồng bộ.
SessionArgs	Một owner trên heap	Cấp phát trong server_run(), giải phóng trong client thread.
TCP socket fd	Sở hữu riêng theo thread	Mỗi lần accept() trả về một descriptor riêng.

Cơ chế bảo vệ dữ liệu chia sẻ.

Đánh giá mô hình Thread-per-Client

Ưu điểm: logic đơn giản, blocking I/O tự nhiên, trạng thái phiên được cách ly. **Giới hạn:** số client chịu ràng buộc bởi MAX_CLIENTS=100 và tài nguyên stack của thread. Khi cần phục vụ hàng nghìn client, hướng mở rộng phù hợp là epoll kết hợp thread pool.

3.1.2 Chuyển đổi Active/Passive Mode

Hybrid FTP hỗ trợ hai cách thiết lập kênh dữ liệu UDP.

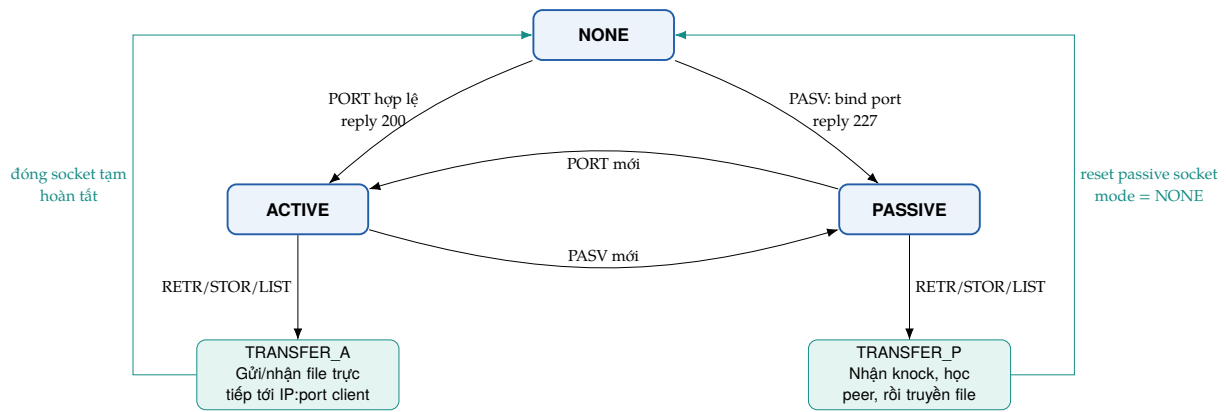
Bảng 8: So sánh Active Mode và Passive Mode.

Đặc điểm	Active Mode (PORT)	Passive Mode (PASV)
Ai mở socket UDP?	Server tạo socket tạm và gửi đến IP:port do client chỉ định.	Server bind một port ngẫu nhiên và chờ datagram đầu tiên từ client.
Hướng khởi tạo	Server → Client	Client → Server
Lệnh FTP	PORT h1, h2, h3, h4, p1, p2	PASV
Phản hồi	200PORTcommandsuccessful.	227EnteringPassiveMode(...)
Phù hợp	Client không bị NAT/firewall chặn kết nối vào.	Client nằm sau NAT/firewall; phổ biến hơn.
Vòng đời	Có thể tái dùng cấu hình IP/port.	Socket passive dùng một lần; phải gọi PASV lại.

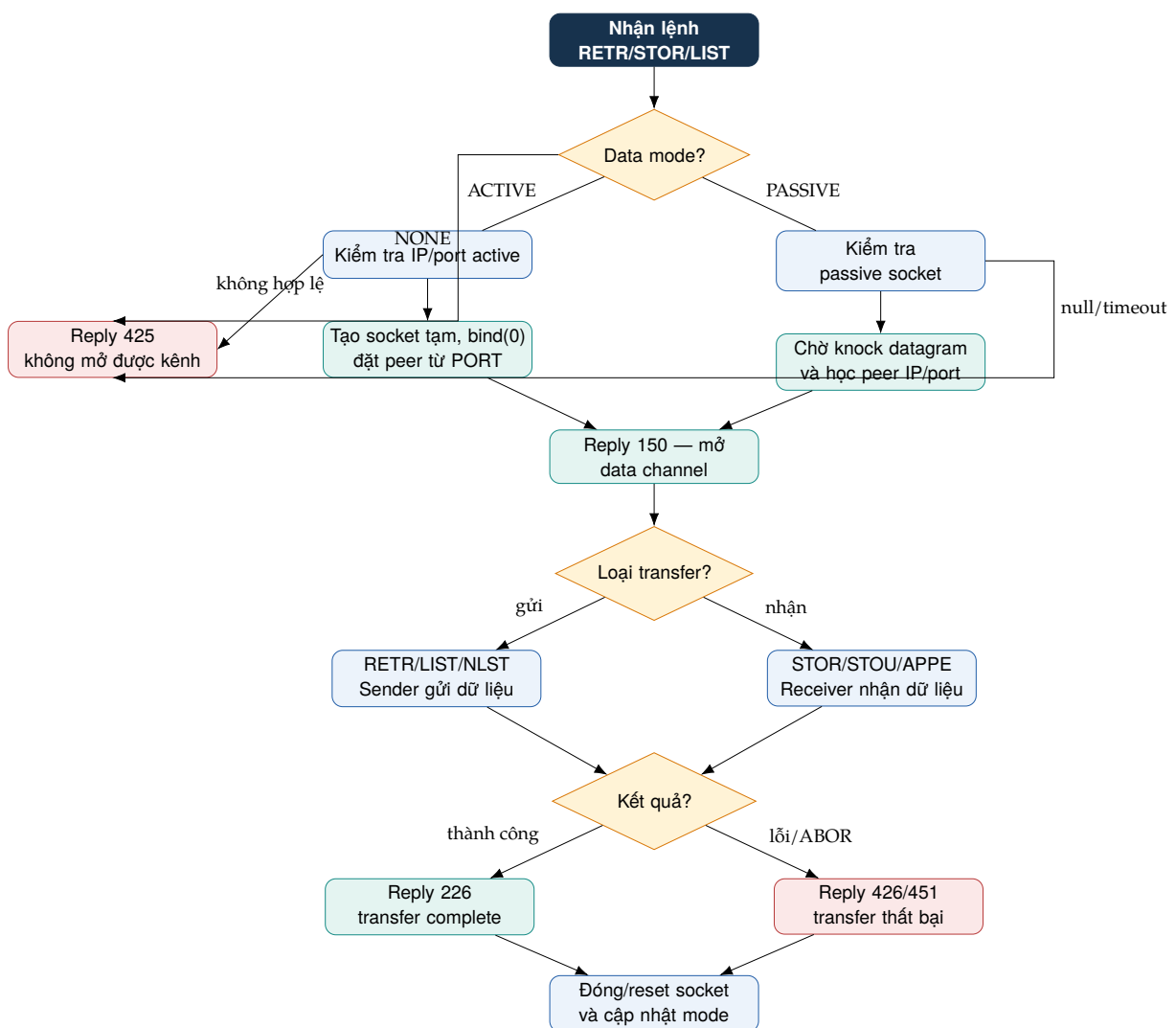
Listing 4: Trạng thái data channel trong một phiên.

```

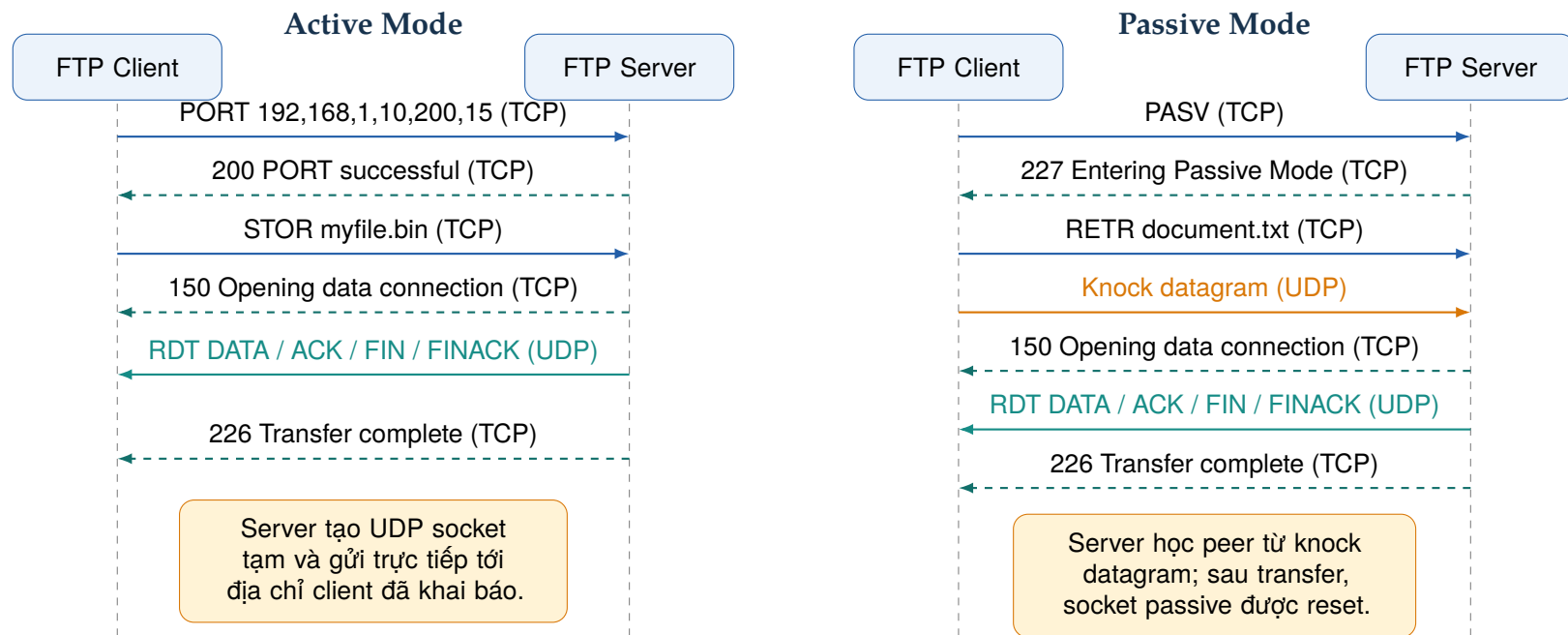
1 enum class DataMode { NONE, ACTIVE, PASSIVE };
2
3 DataMode dataMode = DataMode::NONE;
4 std::string activeIP;
5 uint16_t activePort = 0;
6 std::unique_ptr<UDPSocket> passiveSocket;
```



Hình 6: Máy trạng thái gọn cho Active/Passive Mode.



Hình 7: Quy trình mở và sử dụng data channel cho một lệnh transfer.



Hình 8: Luồng tương tác Active Mode và Passive Mode.

So sánh triển khai trong `open_data_channel`.

Listing 5: Nhánh Active Mode.

```
1 auto *sock = new UDPSocket();
2 sock->open();
3 sock->bind(0);
4 dataSocket = sock;
5 ownsSocket = true;
6 peerIP = session.activeIP;
7 peerPort = session.activePort;
```

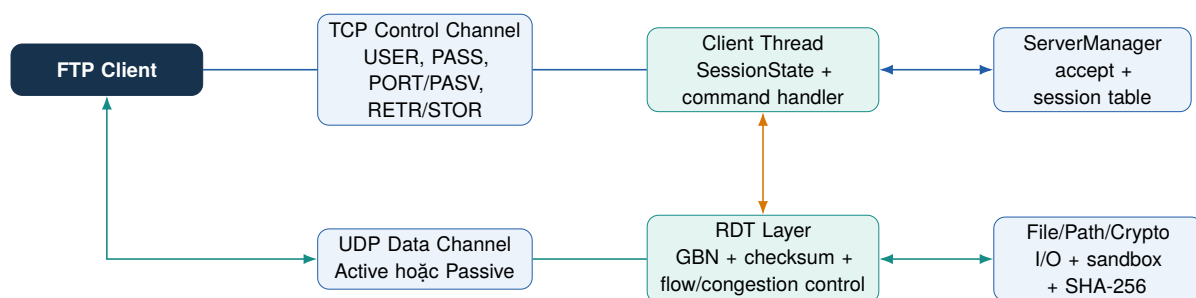
Listing 6: Nhánh Passive Mode.

```
1 dataSocket = session.passiveSocket.get();
2 ownsSocket = false;
3 peerIP = "";
4 peerPort = 0;
5 // recvFrom() learns peer from knock
```

Bảng 9: Dọn dẹp sau transfer.

Chế độ	Thao tác	Lý do
Active	<code>dataSocket->close();</code> <code>deletedataSocket;</code>	Socket tạm được tạo riêng cho transfer.
Passive	<code>passiveSocket.reset();</code> <code>dataMode=NONE;</code>	Passive socket là single-use và phải gọi PASV lại.

3.1.3 Tóm tắt kiến trúc tổng thể



Hình 9: Các lớp chính trong kiến trúc Hybrid FTP.

Kiến trúc Thread-per-Client kết hợp hai chế độ Active/Passive cho phép nhiều client hoạt động đồng thời và chọn cách mở kênh phù hợp với cấu hình mạng. Mutex bảo vệ bảng phiên, trong khi trạng thái và socket của từng phiên được cách ly.

3.2 RDT Sender/Receiver State Machines

Tổng quan kiến trúc

Module RDT gồm hai lớp cửa sổ trượt và hai lớp wrapper cấp cao:

- RDTWindowSender: Go-Back-N, Slow Start và AIMD kiểu TCP Reno rút gọn;
- RDTWindowReceiver: nhận GBN, cumulative ACK và quảng bá receive window;
- RDTSender/RDTReceiver: API sendFile, sendBuffer, receiveFile, receiveBuffer, đồng thời giữ Stop-and-Wait cũ.

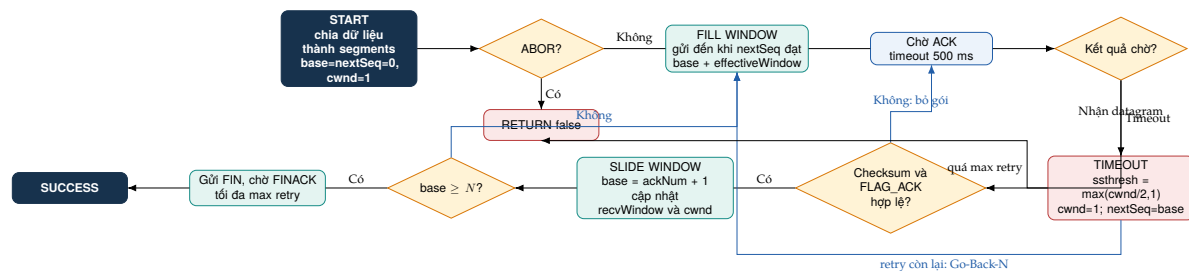
3.2.1 FSM 1 — RDTWindowSender

Biến	Ý nghĩa
base	Sequence nhỏ nhất chưa được ACK; cạnh trái cửa sổ.
nextSeq	Sequence tiếp theo được phép gửi.
cwnd_	Congestion window tính theo số segment.
ssthresh_	Ngưỡng chuyển Slow Start sang Congestion Avoidance.
recvWindow_	Receive window do receiver quảng bá.
retryRnd	Số vòng timeout liên tiếp; reset khi nhận ACK hợp lệ.

Cửa sổ hiệu dụng

$$W_{\text{eff}} = \max(1, \min(\lfloor \text{cwnd} \rfloor, W_{\text{recv}}, W_{\text{max}})) .$$

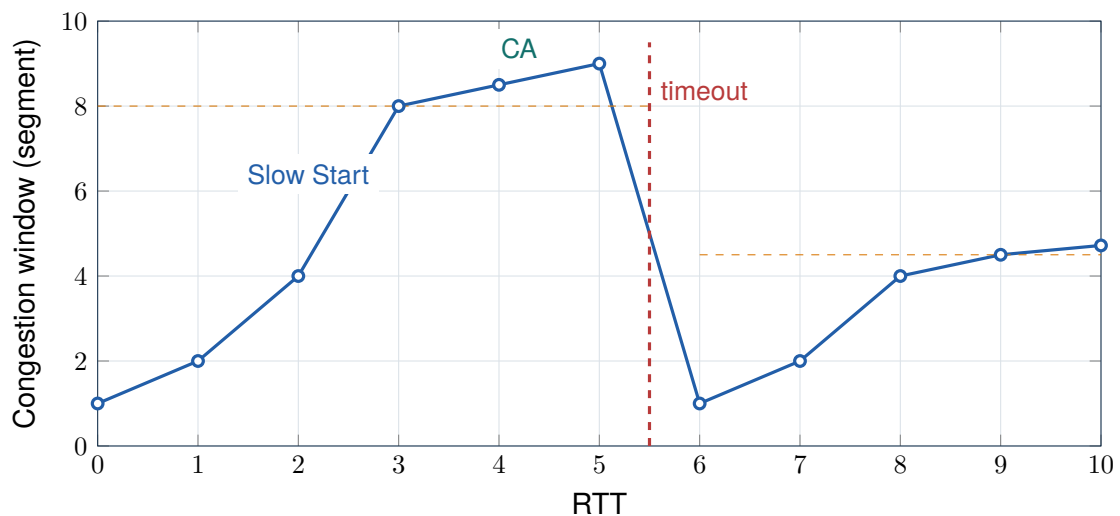
Giới hạn dưới bằng 1 đóng vai trò probe khi receiver quảng bá cửa sổ 0.



Hình 10: FSM đã tinh gọn của RDTWindowSender.

Điều khiển tắc nghẽn

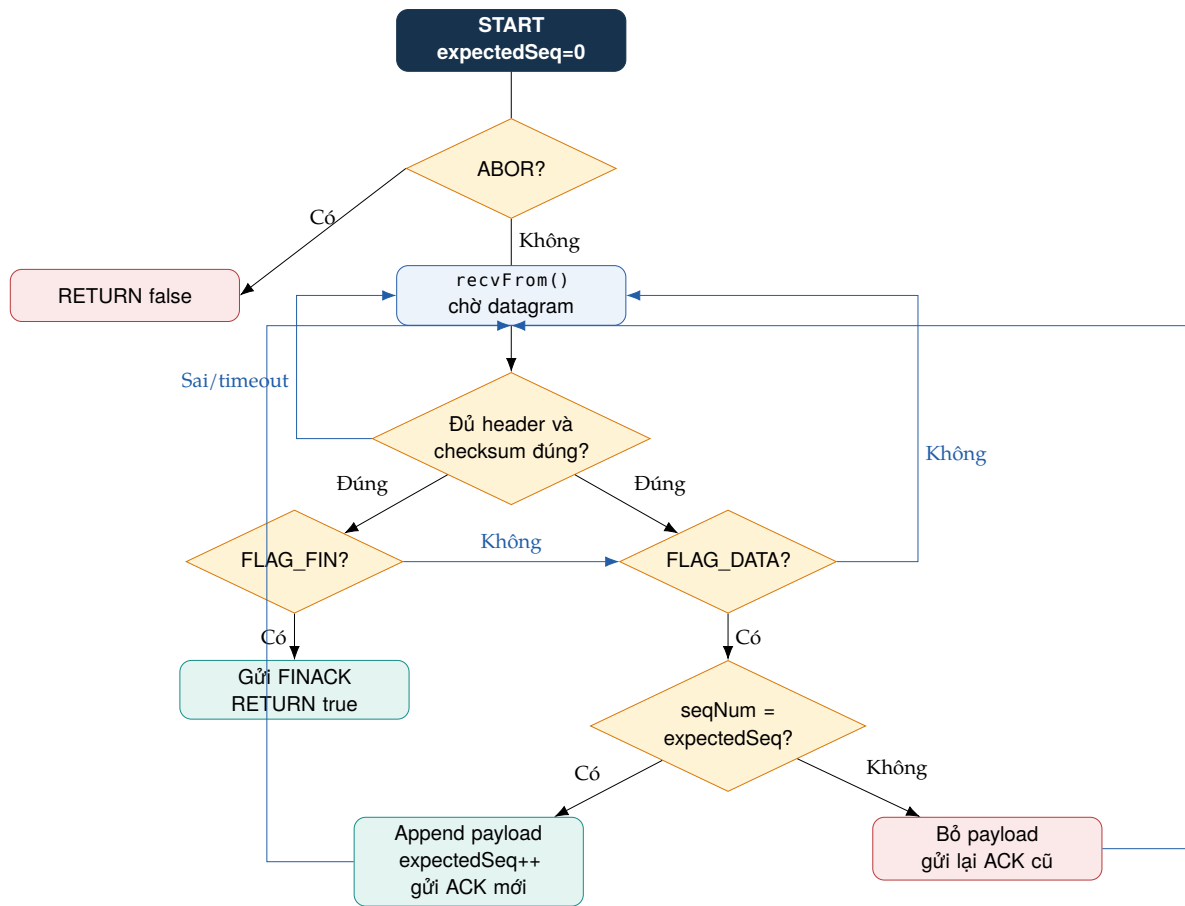
Sự kiện/giai đoạn	Cập nhật
Slow Start, $cwnd < ssthresh$	Mỗi ACK mới: $cwnd \leftarrow cwnd + 1$; xấp xỉ tăng gấp đôi mỗi RTT.
Congestion Avoidance	Mỗi ACK mới: $cwnd \leftarrow cwnd + 1/cwnd$; xấp xỉ tăng 1 segment mỗi RTT.
Timeout	$ssthresh \leftarrow \max(cwnd/2, 1)$, $cwnd \leftarrow 1$, retransmit từ base.
Giới hạn	$cwnd \leftarrow \min(cwnd, W_{\max})$, mặc định $W_{\max} = 32$.



Hình 11: Ví dụ diễn biến cwnd: tăng nhanh, chuyển sang AIMD và giảm sau timeout.

3.2.2 FSM 2 — RDTWindowReceiver

Biến	Ý nghĩa
expectedSeq_	Sequence tiếp theo đang chờ; cũng là số segment đã nhận đúng.
advertisedWindow_	Kích thước buffer được quảng bá cho sender.

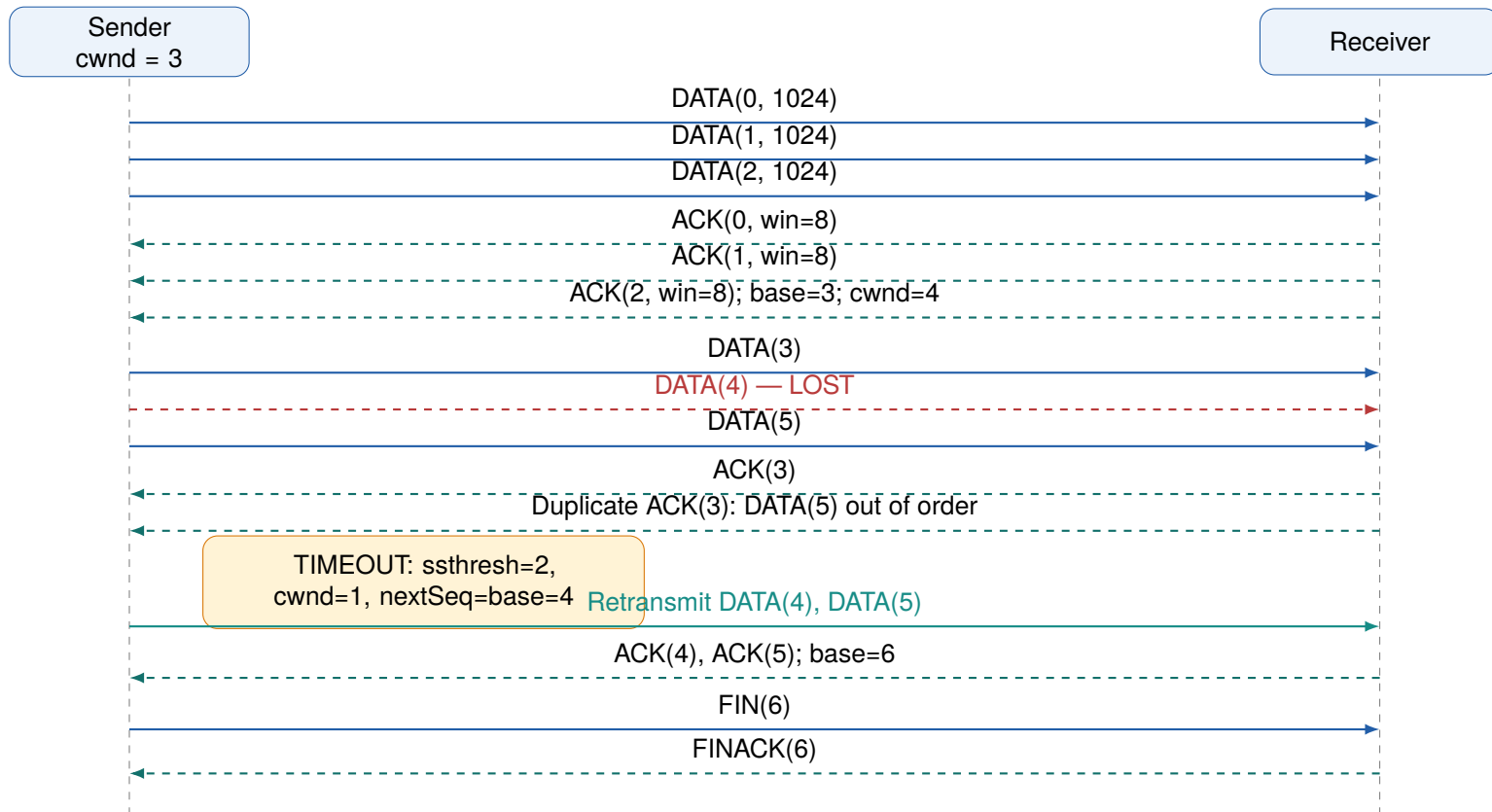


Hình 12: FSM của RDTWindowReceiver với cumulative ACK.

Khi nhận đúng $\text{seqNum} == \text{expectedSeq}$, receiver append payload, tăng expectedSeq và gửi ACK có $\text{ackNum} = \text{expectedSeq} - 1$. ACK còn mang $\text{windowSize} = \text{advertisedWindow}$. Khi nhận gói lệch thứ tự, receiver không buffer gói mà gửi lại ACK cuối; sender sau đó timeout và Go-Back-N từ base.

Bảng 10: Stop-and-Wait và Sliding Window.

Tính năng	Stop-and-Wait	Sliding Window
Gói in-flight	Tối đa 1	Tối đa effectiveWindow .
Congestion control	Không	Slow Start + AIMD.
Flow control	Không	Receive window được quảng bá.
Hiệu năng	Một gói mỗi RTT	Nhiều gói mỗi RTT.
Mục đích	Demo/gói nhỏ, tương thích cũ	Truyền tệp thực tế.



Hình 13: Ví dụ Go-Back-N khi segment 4 bị mất.

4 Task Assignment Matrix — Phân công công việc

Người phụ trách tổng hợp: Dev 3. Phạm vi: Giai đoạn 1 đến Giai đoạn 4.

4.1 Phân công tổng quan theo giai đoạn

Bảng 11: Ma trận phân công theo tiến độ dự án.

Ngày	Công việc	D1	D2	D3	File chính
Giai đoạn 1: Khởi tạo và nền tảng (Ngày 1–3)					
1	Khởi tạo repository Git và cấu trúc thư mục	✓	✓	✓	CMakeLists.txt, .gitignore
1	Thông nhất chuẩn code C++	✓	✓	✓	–
1	Thiết kế định dạng gói tin TCP	✓	–	–	–
1	Thiết kế Custom UDP Header	–	✓	–	src/rdt/CustomUDPHeader.h
2–3	TCP Server Socket và lắng nghe kết nối	✓	–	–	src/server/main_server.cpp
2–3	Xác thực USER, PASS, QUIT	✓	–	–	src/server/Session.cpp
2–3	UDP socket và gửi nhận gói thơ	–	✓	–	src/rdt/UDPSocket.cpp/.h
2–3	Serialize custom UDP header	–	✓	–	src/rdt/CustomHeader.cpp
2–3	Khung giao diện Client CLI	–	–	✓	src/client/ClientCLI.cpp/.h
2–3	Kiến trúc đa luồng trên server	–	–	✓	src/server/ServerManager.cpp/.h
Giai đoạn 2: Xây dựng core logic (Ngày 4–7)					
4–5	Bộ phân tích cú pháp CommandParser	✓	–	–	src/common/CommandParser.cpp/.h
4–5	Lệnh thư mục PWD, CWD, LIST, MKD, RMD	✓	–	–	src/server/Session.cpp
4–5	Hệ thống mã phản hồi 1xx–5xx	✓	–	–	src/server/Session.cpp
4–5	Active/Passive Mode: PORT, PASV	–	–	✓	src/server/Session.cpp
4–5	File I/O binary và ASCII	–	–	✓	src/common/FileHandler.cpp/.h
4–5	PathManager và sandbox đường dẫn	–	–	✓	src/common/PathManager.cpp/.h
6–7	RDT: timeout, retransmit, ACK	–	✓	–	src/rdt/ReliableTransfer.cpp/.h
6–7	Stop-and-Wait	–	✓	–	src/rdt/ReliableTransfer.cpp
6–7	Hỗ trợ debug mạng	✓	✓	–	–
6–7	Tích hợp SHA-256	–	–	✓	src/common/CryptoHash.cpp/.h
Giai đoạn 3: Tích hợp và kiểm thử (Ngày 8–10)					

Còn tiếp ở trang sau

Bảng 11 — tiếp theo

Ngày	Công việc	D1	D2	D3	File chính
8	TYPE {A I}, MODE {S}	✓	—	—	src/server/Session.cpp
8	Điều phối RETR trên control channel	✓	—	—	src/server/Session.cpp
8	Điều phối STOR trên control channel	✓	—	—	src/server/Session.cpp
8	Chia file thành chunks và sendFile	—	✓	—	src/rdt/ReliableTransfer.cpp
8	Nhận chuỗi gói và receiveFile	—	✓	—	src/rdt/ReliableTransfer.cpp
8	FLAG_FIN cho gói cuối	—	✓	—	src/rdt/CustomUDPHeader.h
8	Mở UDP data channel	—	—	✓	src/server/Session.cpp
8	sendFileOverDataChannel	—	—	✓	src/server/Session.cpp
8	receiveFileOverDataChannel	—	—	✓	src/server/Session.cpp
8	Tự động tính hash sau RETR/STOR	—	—	✓	src/server/Session.cpp
8	Test E2E USER-PASS-PASV-STOR-HASH-RETR	—	—	✓	src/common/test_dev3.sh
9	Sliding Window / Go-Back-N	—	✓	—	src/rdt/SlidingWindow.cpp/.h
9	Flow Control	—	✓	—	src/rdt/SlidingWindow.cpp
9	Congestion Control: Slow Start + AIMD	—	✓	—	src/rdt/SlidingWindow.cpp
9	ABOR giữa transfer	✓	—	✓	src/server/Session.cpp
9	STOU với tên file duy nhất	—	—	✓	Session.cpp, PathManager.cpp
9	APPE ghi nối file	—	—	✓	Session.cpp, FileHandler.cpp
9	Client ngắt kết nối đột ngột	✓	✓	—	src/server/Session.cpp
10	Code freeze và sửa lỗi	✓	✓	✓	Toàn bộ repository
10	Build sạch trên clean machine	✓	✓	✓	CMakeLists.txt
10	Test nhiều client đồng thời	—	—	✓	ServerManager.cpp
10	Test ASCII/Binary qua RETR/STOR	—	—	✓	test_dev3.sh
10	Chụp ảnh demo evidence	—	—	✓	docs/
Giai đoạn 4: Tài liệu hóa và bảo vệ (Ngày 11–12)					
11	Section 1: Sequence Diagram vòng đời TCP + UDP	✓	—	—	docs/
11	Section 2.1: TCP Control Packet Format	✓	—	—	docs/
11	Section 2.2: UDP Custom Header Fields	—	✓	—	docs/
11	Section 3.2: RDT Sender/Receiver FSM	—	✓	—	docs/
11	Section 3.1: Thread Dispatch và Active/Passive	—	—	✓	docs/
11	Section 4: Task Assignment Matrix	—	—	✓	docs/
11	Section 5: Self/Peer Evaluation	✓	—	—	docs/
11	Section 6: GenAI Usage Log	—	—	✓	docs/GenAI_Log.md
11	Section 7: Application Demo Evidence	—	—	✓	docs/

Còn tiếp ở trang sau

Bảng 11 — tiếp theo

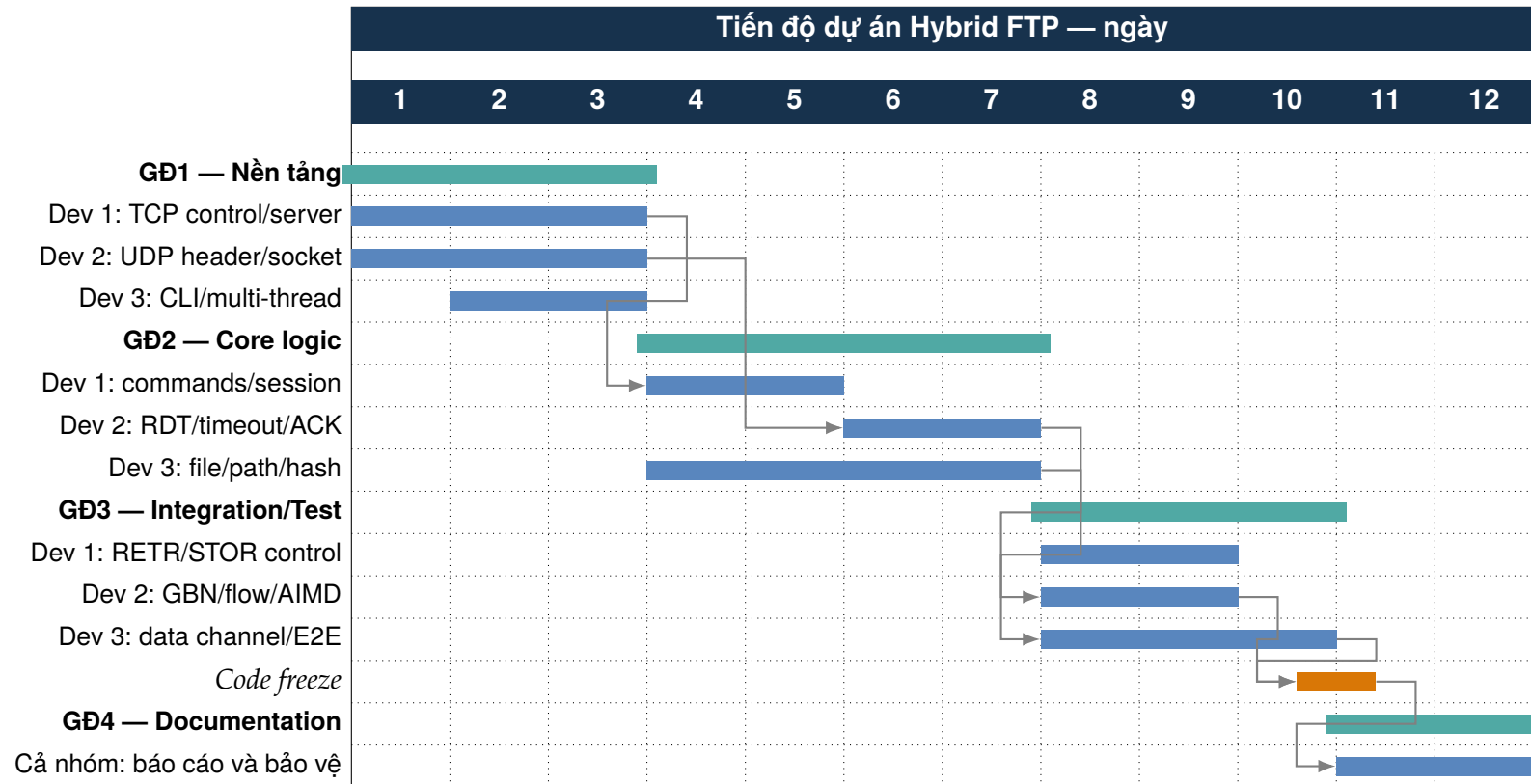
Ngày	Công việc	D1	D2	D3	File chính
12	Diễn tập vấn đáp chéo	✓	✓	✓	–
12	Mô phỏng live coding/debugging	✓	✓	✓	–

4.2 Ma trận phân công theo module/file

Bảng 12: Chủ sở hữu và vai trò của các module.

Module / File	Phát triển chính	Hỗ trợ / Review	Vai trò
src/server/Session.cpp/.h	Dev 1, Dev 3	Dev 2	Control commands, RETR/STOR, data channel, HASH, STOU, APPE.
src/server/ServerManager.cpp/.h	Dev 3	Dev 1	Accept loop, bảng phiên và multi-threading.
src/server/main_server.cpp	Dev 1	–	Entry point FTP server.
src/client/ClientCLI.cpp/.h	Dev 3	Dev 1	CLI, lệnh client và data transfer.
src/client/main_client.cpp	Dev 3	–	Entry point FTP client.
src/common/CommandParser.cpp/.h	Dev 1	–	Phân tích cú pháp lệnh FTP.
src/common/PathManager.cpp/.h	Dev 3	Dev 1	Sandbox, resolve path và unique filename.
src/common/FileHandler.cpp/.h	Dev 3	–	File I/O binary/ASCII và append.
src/common/CryptoHash.cpp/.h	Dev 3	–	SHA-256 kiểm tra toàn vẹn tệp.
src/rdt/CustomUDPHeader.h	Dev 2	–	Cấu trúc header UDP 15 byte.
src/rdt/CustomHeader.cpp	Dev 2	–	Serialize/deserialize và checksum.
src/rdt/UDPSocket.cpp/.h	Dev 2	–	Lớp bọc UDP socket.
src/rdt/ReliableTransfer.cpp/.h	Dev 2	Dev 3	API send/receive file, buffer và Stop-and-Wait.
src/rdt/SlidingWindow.cpp/.h	Dev 2	–	Go-Back-N, flow control và congestion control.
src/rdt/FileTransferClient.cpp	Dev 2	–	Client test RDT.
src/rdt/FileTransferServer.cpp	Dev 2	–	Server test RDT.
test_rdt.sh	Dev 2	Dev 3	Nghiệm thu tự động module RDT.
src/common/test_dev3.sh	Dev 3	–	Kiểm thử E2E.
CMakeLists.txt	Cả nhóm	–	Cấu hình biên dịch CMake.

4.3 Biểu đồ Gantt phân công công việc



Hình 14: Tóm tắt tiến độ và phụ thuộc giữa các nhóm công việc.

4.4 Thống kê đóng góp theo dòng code (ước tính)

Thành viên	Số file chính	Module chính	Ước tính
Dev 1	4	TCP Control, CommandParser, Session control flow	~ 33,33%
Dev 2	7	RDT, UDP Socket, Sliding Window, Congestion Control	~ 33,33%
Dev 3	8	Client CLI, ServerManager, File/Path/Crypto, Data Channel, Tests	~ 33,33%

Cách diễn giải tỷ lệ

Tỷ lệ trên là ước tính dựa trên khối lượng code, độ phức tạp và vai trò kiểm thử; không chỉ dựa vào số dòng. Tất cả thành viên đều tham gia code review và hỗ trợ debug chéo.

5 Self-Assessment & Peer Evaluation

Phần đánh giá sử dụng bằng chứng từ ma trận phân công, phạm vi module, kết quả kiểm thử và quá trình tích hợp. Nhóm không chỉ dựa vào số dòng code, vì các phần thiết kế giao thức, debug mạng và kiểm thử liên module có độ phức tạp khác nhau.

5.1 Tự đánh giá kết quả dự án

Bảng 13: Đánh giá mức độ hoàn thành các mục tiêu chính.

Tiêu chí	Bằng chứng	Tự đánh giá
TCP Control	Framing CRLF, parser, login flow, reply code một/nhiều dòng và session riêng cho từng client.	Hoàn thành tốt
UDP-RDT	Header 15 byte, checksum, DATA/ACK/FIN, Go-Back-N và cumulative ACK.	Hoàn thành tốt
Flow/Congestion Control	Advertised window, effective window, Slow Start và AIMD.	Hoàn thành
Active/Passive Mode	PORT/PASV, passive knock, socket single-use và dọn dẹp sau transfer.	Hoàn thành tốt
File Operations	RETR, STOR, STOU, APPE, LIST/NLST, PathManager và SHA-256.	Hoàn thành tốt
Đa client và kiểm thử	Thread-per-Client, mutex bảng phiên, build sạch và kịch bản E2E.	Hoàn thành
Tài liệu kỹ thuật	Wire format, state machine, sequence diagram và ma trận phân công.	Hoàn thành tốt

Nhận định chung

Sản phẩm đáp ứng mục tiêu xây dựng một FTP server/client lai: TCP được dùng cho điều khiển, còn UDP-RDT đảm nhiệm truyền dữ liệu tin cậy. Các module đã được tích hợp thành luồng E2E và có thể giải thích bằng wire format, FSM và sequence flow trong báo cáo.

5.2 Tự đánh giá theo thành viên

Bảng 14: Tự đánh giá dựa trên phạm vi phụ trách.

Thành viên	Đóng góp chính	Tự đánh giá và hướng cải thiện
Nguyễn Minh Nhật (Dev 1)	TCP control, CommandParser, command/session flow, RETR/STOR orchestration.	Hoàn thành luồng điều khiển và phối hợp tích hợp. Cần bổ sung xác thực mật khẩu thực tế và tăng kiểm thử parser với input biên.
Huỳnh Nhật Hạ (Dev 2)	Custom UDP header, socket wrapper, RDT, Sliding Window, flow/congestion control.	Hoàn thành phần thuật toán mạng cốt lõi và cơ chế truyền lại. Cần mở rộng test loss/reorder có kiểm soát và cân nhắc Fast Retransmit hoặc Selective Repeat.
Lê Hữu Anh Thư (Dev 3)	Client CLI, ServerManager, Active/Passive data channel, file/path/hash và E2E tests.	Hoàn thành tích hợp hai đầu, quản lý tài nguyên và kiểm thử chức năng. Cần tăng test stress nhiều client và hoàn thiện hủy transfer đồng bộ.

5.3 Peer Evaluation

Các thành viên thực hiện review chéo tại điểm giao nhau giữa control channel, data channel và file I/O. Đánh giá chéo tập trung vào khả năng hoàn thành phần việc, chất lượng tích hợp, hỗ trợ debug và mức độ hiểu hệ thống ngoài module cá nhân.

Bảng 15: Tổng hợp đánh giá chéo trong nhóm.

Thành viên được đánh giá	Bằng chứng từ đồng đội	Kết luận
Nguyễn Minh Nhật	Cung cấp control flow ổn định để Dev 2 và Dev 3 tích hợp RDT/data channel; hỗ trợ debug lỗi kết nối và command state.	Hoàn thành tốt
Huỳnh Nhật Hạ	Xây dựng giao diện RDT dùng chung, giải thích được timeout/window/checksum và hỗ trợ kiểm thử truyền file hai chiều.	Hoàn thành tốt
Lê Hữu Anh Thư	Kết nối các module thành client/server chạy được, quản lý path/file/hash và xây dựng kịch bản E2E, multi-client.	Hoàn thành tốt

Nhóm thống nhất tỷ lệ đóng góp ước tính gần tương đương ($\sim 33,33\%$ cho mỗi thành viên). Sự khác biệt về số file không phản ánh trực tiếp khối lượng vì RDT có ít file hơn nhưng thuật toán phức tạp, còn phân tích hợp và kiểm thử trải trên nhiều module.

5.4 Hạn chế và kế hoạch cải tiến

Hạn chế hiện tại	Hướng cải tiến
PASS chưa kiểm tra mật khẩu thực tế	Thêm user database, password hashing và giới hạn số lần đăng nhập sai.
ABOR mới ở mức best-effort	Tách transfer worker hoặc dùng cancellation flag thread-safe để ngắt RDT tức thời.
Go-Back-N truyền lại cả cửa sổ	Bổ sung duplicate ACK, Fast Retransmit hoặc Selective Repeat/SACK.
Thread-per-Client giới hạn khả năng mở rộng	Chuyển sang thread pool kết hợp epoll khi số client lớn.
Kiểm thử mạng chủ yếu ở môi trường cục bộ	Thêm network emulator để kiểm soát loss, delay, jitter, reorder và corruption.
Passive endpoint học từ datagram đầu tiên	Bổ sung token phiên hoặc xác thực peer để giảm nguy cơ datagram ngoài phiên.

Bài học rút ra

Điểm khó nhất không nằm ở từng socket riêng lẻ mà ở ranh giới giữa TCP Control, UDP-RDT và file I/O. Việc thống nhất state transition, quyền sở hữu socket và thời điểm phát reply 125/150/226 giúp giảm lỗi tích hợp và làm cho hành vi hệ thống có thể kiểm thử, giải thích và bảo trì.

6 GenAI Usage & Code Refinement Log

Nhóm sử dụng các mô hình GenAI như công cụ hỗ trợ trong quá trình phân tích, phát triển, rà soát và chuẩn bị nghiệm thu. GenAI không trực tiếp quyết định kiến trúc hoặc thay thế việc triển khai; các thành viên chịu trách nhiệm đánh giá tính phù hợp của từng đề xuất trước khi đưa vào dự án.

6.1 Phạm vi sử dụng GenAI

1. Phân tích yêu cầu, phân công công việc cho các thành viên và đề xuất kiến trúc tổng thể của Hybrid FTP.
2. Giải thích socket API, TCP Control Channel và UDP Data Channel.
3. Review tiến độ, đối chiếu yêu cầu và phát hiện các chức năng còn thiếu.
4. Gợi ý hướng debug khi chạy trên WSL và trên hai máy tính vật lý.
5. Chuẩn bị kiến thức cho phần vấn đáp và live coding.
6. Đề xuất test case, checklist demo và cấu trúc trình bày báo cáo.

6.2 Lịch sử trao đổi theo thành viên

Các liên kết dưới đây lưu lại những cuộc hội thoại hoặc tài liệu làm việc chính của từng thành viên với ChatGPT, Gemini và Claude.

6.2.1 Dev 1 — Nguyễn Minh Nhật

- ▷ <https://chatgpt.com/share/6a7c88a0-14d0-83ec-b721-3a6efa990cae>
- ▷ <https://chatgpt.com/share/6a7c88ba-99b8-83ec-842a-df7ef4f582a1>
- ▷ <https://chatgpt.com/share/6a7c87cf-a19c-83ec-9da7-ca0320501074>
- ▷ <https://chatgpt.com/share/6a7c87b3-2310-83ec-8041-1fb69f4306be>

6.2.2 Dev 2 — Huỳnh Nhật Hạ

- ▷ <https://share.gemini.google/RQUIIzmQt1D7>
- ▷ <https://claude.ai/share/62c78f62-0ff6-40e0-8ad0-345e17383d79>
- ▷ <https://share.gemini.google/6QS1Dnz4EsNV>
- ▷ <https://claude.ai/share/cfe19072-7ef5-4afd-a1af-ccfd9bd8bd9c>

6.2.3 Dev 3 — Lê Hữu Anh Thư

- ▷ <https://claude.ai/share/75502be9-fadc-4b96-bf7c-abac0e3aace0>

- ▷ <https://claude.ai/share/ab259b22-11f1-415d-bd83-3b459e1f19bc>
- ▷ https://docs.google.com/document/d/1bR54Q9yTvY6_FoG4fCUY-YfzSWx0sihchM1SZ76RfwY/edit?usp=sharing

6.3 Nguyên tắc kiểm chứng và tinh chỉnh code

Trách nhiệm học thuật và kỹ thuật

GenAI không thay thế trách nhiệm thiết kế, lập trình và kiểm thử của sinh viên. Mọi đề xuất đều được đối chiếu với đặc tả dự án, code hiện tại và kết quả chạy thực tế trước khi sử dụng. Khi một gợi ý làm thay đổi giao diện module, trạng thái giao thức hoặc quy trình truyền file, nhóm tiếp tục review chéo và chạy lại các test liên quan trước khi chấp nhận thay đổi.

7 Application Demo Evidence

Minh chứng nghiệm thu — Hybrid FTP System

Người phụ trách: Dev 3 **Môi trường:** hai máy tính riêng biệt (Server và Client).
Chương này mô tả quy trình chuẩn bị, thao tác kiểm thử và ảnh chụp bằng chứng cho các chức năng truyền file, kiểm tra toàn vẹn, quản lý phiên và xử lý đồng thời. Các địa chỉ IP và timestamp trong lệnh hoặc log chỉ là ví dụ; khi nghiệm thu cần thay bằng giá trị thực của hai máy.

7.1 Tổng quan các ca kiểm thử

Hybrid FTP cần thể hiện bảy ca kiểm thử chính để chứng minh toàn bộ tính năng. Tất cả các ca được thực hiện trên hai máy tính thực trong cùng mạng, không dùng địa chỉ loopback.

Bảng 16: Danh sách ca kiểm thử và bằng chứng cần thu thập.

STT	Ca kiểm thử	Mục đích	Bằng chứng cần có
1	Upload file thành công	Chứng minh STOR, RDT và thao tác ghi file.	Ảnh terminal, log server và file đã nhận.
2	Download file thành công	Chứng minh RETR, RDT và thao tác gửi file.	Ảnh terminal, log server và kích thước file.
3	So sánh SHA-256	Chứng minh tính toàn vẹn dữ liệu.	Mã băm trước và sau truyền khớp hoàn toàn.
4	Danh sách phiên client	Chứng minh multi-threading và quản lý phiên.	IP, port, thread và trạng thái client.
5	Nhiều client đồng thời	Chứng minh server xử lý nhiều client.	Log ít nhất ba client cùng upload/download.
6	File ASCII	Chứng minh hỗ trợ TYPE A.	Reply, log và nội dung file sau truyền.
7	File binary	Chứng minh hỗ trợ TYPE I.	Reply, kích thước, hash và dữ liệu nhị phân.

7.2 Chuẩn bị môi trường hai máy tính

7.2.1 Sơ đồ kết nối mạng

Hai máy tính phải nằm trong cùng mạng LAN hoặc kết nối trực tiếp. Cần xác định địa chỉ IP của máy Server trước khi chạy bất kỳ ca kiểm thử nào.



Hình 15: Kết nối kiểm thử giữa hai máy tính vật lý.

Thay 192.168.1.100 bằng IP thực của Server. Có thể dùng `ipaddr` trên Linux hoặc `ipconfig` trên Windows để xác định địa chỉ.

7.2.2 Kiểm tra kết nối mạng

Trên máy Client, kiểm tra khả năng truy cập Server và TCP port 2121:

Listing 7: Kiểm tra kết nối từ máy Client.

```
1 ping 192.168.1.100
2 telnet 192.168.1.100 2121
3 # Hoac dung netcat:
4 nc -zv 192.168.1.100 2121
```

7.2.3 Biên dịch dự án trên cả hai máy

Thực hiện từ thư mục gốc của dự án trên máy Server:

Listing 8: Biên dịch trên máy Server.

```
1 rm -rf build
2 mkdir build
3 cd build
4 cmake .. -DCMAKE_BUILD_TYPE=Release
5 cmake --build . -- -j$(nproc)
6 ls -lh ftp_server ftp_client
7 cd ..
```

Lặp lại quy trình trên máy Client; tối thiểu phải tạo được executable `build/ftp_client`. Kết quả kỳ vọng là quá trình CMake hoàn tất không lỗi và các executable có quyền thực thi.

7.2.4 Chuẩn bị dữ liệu test

Tạo thư mục dữ liệu và hai file mẫu trên máy phù hợp với chiều truyền của ca kiểm thử. Các lệnh sau được chạy từ thư mục gốc dự án:

Listing 9: Tạo dữ liệu ASCII và binary mẫu.

```
1 mkdir -p data_test
2
3 cat > data_test/sample_text.txt <<'EOF'
4 Hybrid FTP Server - File Transfer Protocol Implementation
5 Development Group: Dev1, Dev2, Dev3
6 Date: August 2026
7 Testing ASCII mode transfer
8 EOF
9
10 python3 -c "import os; open('data_test/sample_binary.bin', 'wb').write(os.urandom(1024*1024))"
11 sha256sum data_test/sample_text.txt > data_test/sample_text.sha256
12 sha256sum data_test/sample_binary.bin > data_test/sample_binary.sha256
13 ls -lh data_test/sample_*
```

7.3 Ca kiểm thử 1: Upload file thành công (STOR + RDT)

7.3.1 Mục đích

- Client gửi file đến Server bằng lệnh STOR.
- RDT truyền dữ liệu tin cậy qua UDP.
- Server ghi file thành công xuống đĩa và trả reply 226.

7.3.2 Các bước thực hiện

Khởi chạy Server và lưu log vào thư mục evidence:

Listing 10: Khởi chạy Server cho ca upload.

```
1 mkdir -p evidence
2 ./build/ftp_server 2121 2>&1 | tee evidence/server_upload_test.log
```

Trên máy Client, tạo file binary, lưu hash gốc rồi upload:

Listing 11: Upload file binary từ Client.

```
1 mkdir -p data_test/upload
2 python3 -c "import os; open('data_test/upload/sample_binary.bin', 'wb').write(os.urandom
  (1024*1024))"
3 sha256sum data_test/upload/sample_binary.bin \
4 > data_test/upload/sample_binary.sha256.client
5
6 cd data_test/upload
7 ../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
8 user testuser
9 pass testpass
10 type I
11 pasv
12 stor sample_binary.bin
```

```

13 hash sample_binary.bin
14 quit
15 COMMANDS
16 cd ../../

```

Trên Server, kiểm tra file đã nhận, kích thước chính xác và SHA-256:

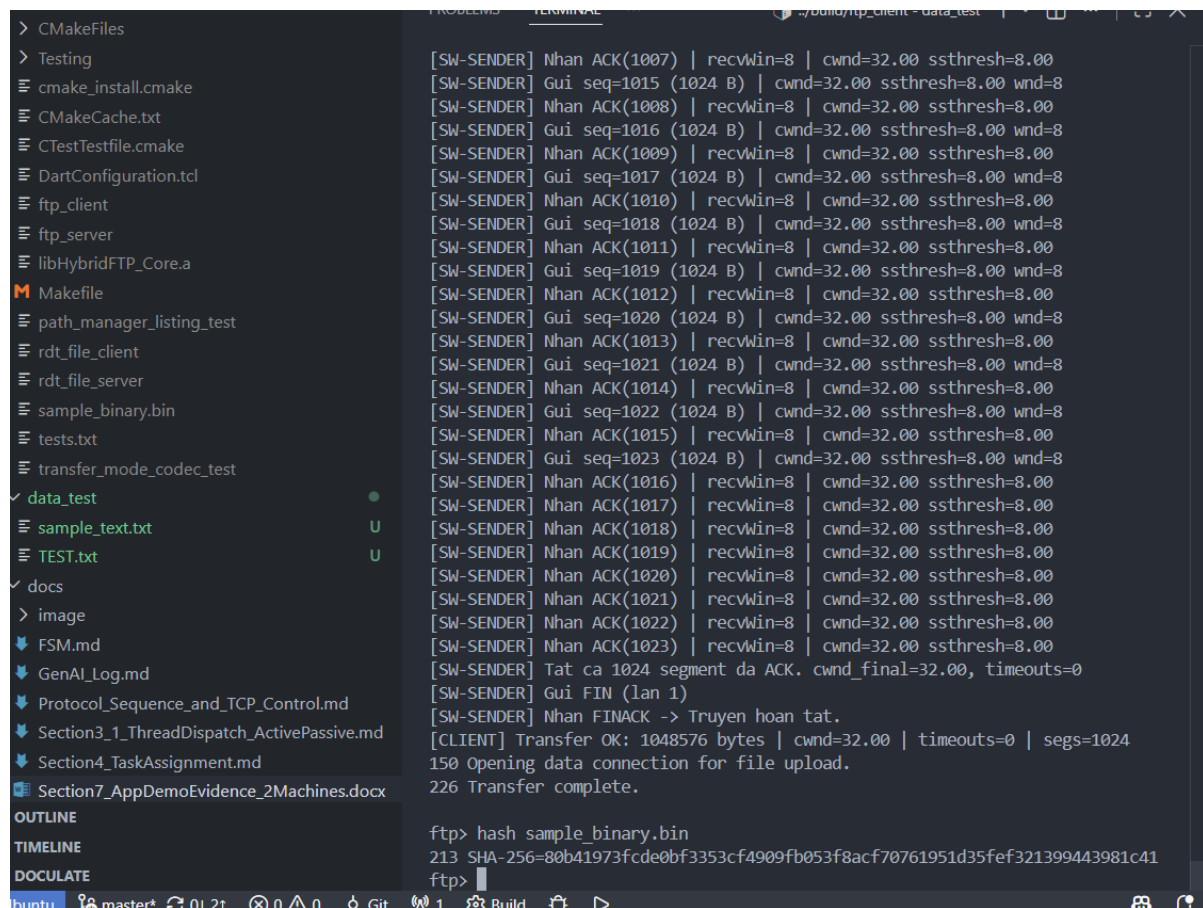
Listing 12: Xác nhận file upload trên Server.

```

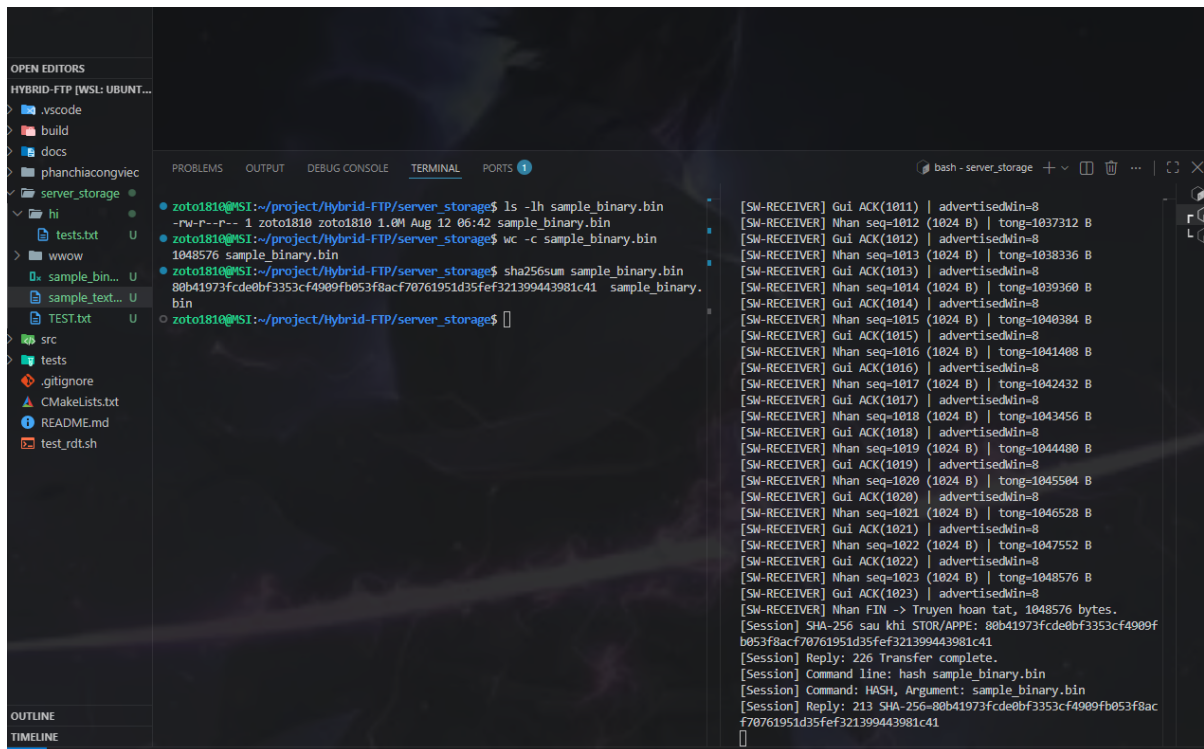
1 ls -lh data_test/sample_binary.bin
2 wc -c data_test/sample_binary.bin
3 sha256sum data_test/sample_binary.bin
4 # Kích thước kỳ vọng: 1048576 byte; hash phải khớp với Client.

```

7.3.3 Bảng chứng thu được



Hình 16: Log Server nhận các segment RDT, FIN và hoàn tất phiên upload.



Hình 17: Terminal Client/Server xác nhận file upload tồn tại và có mã SHA-256.

7.4 Ca kiểm thử 2: Download file thành công (RETR + RDT)

7.4.1 Mục đích

- Server gửi file đến Client bằng lệnh RETR.
- RDT vận chuyển dữ liệu tin cậy trên kênh UDP.
- Client nhận file đúng nội dung và kích thước.

7.4.2 Các bước thực hiện

Đảm bảo file nguồn tồn tại trên Server, sau đó khởi chạy server:

Listing 13: Chuẩn bị Server cho ca download.

```
1 ls -lh data_test/sample_binary.bin
2 # Neu chua co file:
3 python3 -c "import os; open('data_test/sample_binary.bin','wb').write(os.urandom(1024*1024))"
4
5 mkdir -p evidence
6 ./build/ftp_server 2121 2>&1 | tee evidence/server_download_test.log
```

Trên Client, chuyển vào thư mục nhận và thực hiện download:

Listing 14: Download file bằng RETR.

```
1 mkdir -p data_test/download
2 cd data_test/download
```



```

3 ../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
4 user testuser
5 pass testpass
6 type I
7 pasv
8 retr sample_binary.bin
9 quit
10 COMMANDS
11
12 ls -lh sample_binary.bin
13 wc -c sample_binary.bin
14 cd ../../

```

Log kỳ vọng chứa reply 150, thông tin RDT sender, số byte đã gửi và reply 226. Phía Client phải hiển thị tiến trình download, reply 226 và file data_test/download/sample_binary.bin có kích thước 1,048,576 byte.

7.4.3 Bảng chứng thu được

```

[Session] Command line: PASV
[Session] Command: PASV, Argument:
[Session] Reply: 227 Entering Passive Mode (172,20,10,3,186,148).
[Session] Command line: LIST
[Session] Command: LIST, Argument:
[Session] Reply: 150 Opening data connection for directory list.
[RD-T-SENDER] sendBuffer: 56 bytes
[SW-SENDER] Bat dau gui 1 segment (MSS=1024, cwnd=1.00, ssthresh=8.00)
[SW-SENDER] Gui seq=0 (56 B) | cwnd=1.00 ssthresh=8.00 wnd=1
[SW-SENDER] Nhan ACK(0) | recvWin=8 | cwnd=1.00 ssthresh=8.00
[SW-SENDER] Tat ca 1 segment da ACK. cwnd_final=2.00, timeouts=0
[SW-SENDER] Gui FIN (lan 1)
[SW-SENDER] Nhan FINACK -> Truyen hoan tat.
[Session] Reply: 226 Directory transfer complete.
[Session] Command line: cwd hi
[Session] Command: CWD, Argument: hi
[Session] Reply: 250 Directory changed successfully.
[Session] Command line: PASV
[Session] Command: PASV, Argument:
[Session] Reply: 227 Entering Passive Mode (172,20,10,3,190,63).
[Session] Command line: RETR tests.txt
[Session] Command: RETR, Argument: tests.txt
[Session] Reply: 150 Opening data connection for file download.
[RD-T-SENDER] sendBuffer: 24 bytes
[SW-SENDER] Bat dau gui 1 segment (MSS=1024, cwnd=1.00, ssthresh=8.00)
[SW-SENDER] Gui seq=0 (24 B) | cwnd=1.00 ssthresh=8.00 wnd=1
[SW-SENDER] Nhan ACK(0) | recvWin=8 | cwnd=1.00 ssthresh=8.00
[SW-SENDER] Tat ca 1 segment da ACK. cwnd_final=2.00, timeouts=0
[SW-SENDER] Gui FIN (lan 1)
[SW-SENDER] Nhan FINACK -> Truyen hoan tat.
[Session] SHA-256 sau khi RETR: aaf4f2efcef19afacd23d82dc0c3d81d5e4b94ba9a7f1c274d5cafd515d7f460
[Session] Reply: 226 Transfer complete.

```

Hình 18: Log Server xử lý RETR và hoàn tất truyền file.

```

EXPLORED  ...  dding.md  • Section7_AppDemoEvidence_2Machines.docx  tests.txt
FTP_PROJECT [WSL: UBUNTU]
> .vscode
> build
  > CMakeFiles
  > Testing
  = cmake_install.cmake
  = CMakeCache.txt
  = CTestTestfile.cmake
  = DartConfiguration.tcl
  = ftp_client
  = ftp_server
  = libHybridFTP_Core.a
  = Makefile
  = path_manager_listing_test
  = rdt_file_client
  = rdt_file_server
  = sample_binary.bin
  = tests.txt
  = transfer_mode_codec_test
> docs
  > image
  = FSM.md
  = GenAI_Log.md
  = Protocol_Sequence_and_TCP_Control.md
  = Section3_1_ThreadDispatch_ActivePassive.md
  = Section4_TaskAssignment.md
  = Section7_AppDemoEvidence_2Machines.docx
  = Section7_AppDemoEvidence.md
  = Technical_Report.docx
  = UDPHeader.md
> OUTLINE
> TIMELINE
> DOCULATE

build > tests.txt
1 Xin Chao Thu Le Dep Gail!

PROBLEMS  OUTPUT  TERMINAL  ...  /ftp_client - build
thhuule@LAPTOP-83RK0RML:/mnt/c/Users/ACER/FTP_PROJECT/build$ ./ftp_client 172.20.10.3 2121
227 Entering Passive Mode (172,20,10,3,179,197).
ftp> LIST
[SW-RECEIVER] Nhan seq=0 (0 B) | tong=0 B
[SW-RECEIVER] Gui ACK(0) | advertisedwin=8
[SW-RECEIVER] Nhan FIN -> Truyen hoan tat, 0 bytes.
150 Opening data connection for directory list.
226 Directory transfer complete.

ftp> MKD hi
257 Directory created successfully.
ftp> retr tests.txt
550 File not found or access denied.
ftp> list
[SW-RECEIVER] Nhan seq=0 (56 B) | tong=56 B
[SW-RECEIVER] Gui ACK(0) | advertisedwin=8
[SW-RECEIVER] Nhan FIN -> Truyen hoan tat, 56 bytes.
150 Opening data connection for directory list.
name=hi, type=directory, size=0, permissions=rwxr-xr-x
226 Directory transfer complete.

ftp>
ftp> cwd hi
250 Directory changed successfully.
ftp> retr tests.txt
[CLIENT] Nhan file 'tests.txt' <- 'tests.txt' tu 172.20.10.3:48703
[SW-RECEIVER] Nhan seq=0 (24 B) | tong=24 B
[SW-RECEIVER] Gui ACK(0) | advertisedwin=8
[SW-RECEIVER] Nhan FIN -> Truyen hoan tat, 24 bytes.
[CLIENT] SHA-256 sau RETR: aaf4f2efcef19afacd23d82dc0c3d81d5e4b94ba9a7f1c274
d5cafd515d7f460 (24 bytes)
150 Opening data connection for file download.
226 Transfer complete.

```

Hình 19: Client nhận file qua RDT và nhận reply 226 Transfer complete.

```

ls: cannot access './build/tests.txt': No such file or directory
thhuule@LAPTOP-83RK0RML:/mnt/c/Users/ACER/FTP_PROJECT$ ls -lh ./build/tests.txt
-rwxrwxrwx 1 thhuule thhuule 24 Aug 12 06:06 ./build/tests.txt
thhuule@LAPTOP-83RK0RML:/mnt/c/Users/ACER/FTP_PROJECT$ wc -c ./build/tests.txt
24 ./build/tests.txt
thhuule@LAPTOP-83RK0RML:/mnt/c/Users/ACER/FTP_PROJECT$

```

Hình 20: Kiểm tra file download tồn tại và có đúng kích thước 1,048,576 byte.

7.5 Ca kiểm thử 3: Kiểm tra SHA-256 toàn vẹn dữ liệu

7.5.1 Mục đích

- Chứng minh file không bị hỏng trong quá trình truyền qua RDT.
- So sánh SHA-256 trước và sau transfer; hai giá trị phải khớp hoàn toàn.
- Kiểm tra lệnh HASH hoạt động đúng.

7.5.2 Các bước thực hiện

Tính hash gốc trên Client, upload file, yêu cầu Server trả hash rồi kiểm tra trực tiếp trên Server:

Listing 15: Quy trình kiểm tra SHA-256.

```

1 # Client: hash goc
2 sha256sum data_test/upload/sample_binary.bin
3
4 cd data_test/upload
5 ../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
6 user testuser
7 pass testpass
8 type I
9 pasv
10 stor sample_binary.bin
11 hash sample_binary.bin
12 quit
13 COMMANDS
14 cd ../../
15
16 # Server: hash của file đã nhận
17 sha256sum data_test/sample_binary.bin

```

Kết quả đạt khi hash gốc phía Client, reply 213 của lệnh HASH và hash tính trực tiếp trên Server giống nhau 100%. Nếu khác nhau, ca kiểm thử thất bại và cần kiểm tra lại checksum, thứ tự segment và thao tác ghi file.

7.5.3 Bảng chứng thu được

```

thhuule@LAPTOP-8JRK0RML:/mnt/c/Users/ACER/FTP_PROJECT$ sha256sum /mnt/c/Users/ACER/FTP_PROJECT/data_test/sample_binary.bin
80b41973fcde0bf3353cf4909fb053f8acf70761951d35fef321399443981c41 /mnt/c/Users/ACER/FTP_PROJECT/data_test/sample_binary.bin
○ thhuule@LAPTOP-8JRK0RML:/mnt/c/Users/ACER/FTP_PROJECT$

```

Hình 21: SHA-256 của file gốc trên Client trước khi upload.

```

[SW-SENDER] Gửi FIN (lan 1)
[SW-SENDER] Nhận FINACK -> Truyền hoàn tất.
[CLIENT] Transfer OK: 1048576 bytes | cwnd=32.00 | timeouts=0 | segs=1024
150 Opening data connection for file upload.
226 Transfer complete.

ftp> hash sample_binary.bin
213 SHA-256=80b41973fcde0bf3353cf4909fb053f8acf70761951d35fef321399443981c41

```

Hình 22: Reply 213 của lệnh HASH sau khi Client upload file.

```

zoto181@PS1:~/project/hybrid-FTP/server_storage$ echo "Client hash (gốc): 80b41973fcd0bf3353cf4909fb053f8acf70761951d35fef321399443981c41"
Client hash (gốc): 80b41973fcd0bf3353cf4909fb053f8acf70761951d35fef321399443981c41
zoto181@PS1:~/project/hybrid-FTP/server_storage$ echo "=== HASH COMPARISON ==="
=== HASH COMPARISON ===
zoto181@PS1:~/project/hybrid-FTP/server_storage$ echo "Client hash (gốc): 80b41973fcd0bf3353cf4909fb053f8acf70761951d35fef321399443981c41"
Client hash (gốc): 80b41973fcd0bf3353cf4909fb053f8acf70761951d35fef321399443981c41
zoto181@PS1:~/project/hybrid-FTP/server_storage$ echo "Server hash (nhận): $(sha256sum sample_binary.bin | awk '{print $1}')"
Server hash (nhận): 80b41973fcd0bf3353cf4909fb053f8acf70761951d35fef321399443981c41
zoto181@PS1:~/project/hybrid-FTP/server_storage$

```

Hình 23: So sánh SHA-256 giữa file gốc và file được lưu trên Server.

7.6 Ca kiểm thử 4: Danh sách phiên kết nối Client

7.6.1 Mục đích

- Server theo dõi các session đang kết nối.
- Mỗi client được phục vụ bởi một thread riêng.
- Bảng ClientRecord lưu IP, port và trạng thái của từng client.

7.6.2 Các bước thực hiện

Khởi chạy Server với log theo dòng, sau đó mở ba terminal Client và duy trì ba kết nối đồng thời:

Listing 16: Khởi chạy Server để theo dõi session.

```

1 mkdir -p evidence
2 stdbuf -oL ./build/ftp_server 2121 2>&1 \
3 | tee evidence/session_tracking.log

```

Trong ba terminal Client, đăng nhập cùng tài khoản và lần lượt thực hiện PWD, LIST hoặc TYPE I. Khi các kết nối còn mở, lọc log trên Server:

Listing 17: Lọc thông tin session trên Server.

```

1 grep -iE 'Thread|connection|Client|Session' \
2 evidence/session_tracking.log

```

Bảng 17: Bảng session kỳ vọng khi ba client kết nối đồng thời.

Slot	IP	Port	Thread	Trạng thái	User/Data mode
0	192.168.1.101	48523	#1	ACTIVE	testuser / TYPE I
1	192.168.1.101	48524	#2	ACTIVE	testuser / PASV
2	192.168.1.101	48525	#3	ACTIVE	testuser / TYPE I

```
[Server] Hybrid FTP control server listening on TCP port 2121
[Server] Thread #1 assigned to client 172.20.10.6:48514 (fd=4)
[Thread #1] Session started: 172.20.10.6:48514, fd=4
[Session] Reply: 220 Hybrid FTP service ready.
[Server] Thread #2 assigned to client 172.20.10.10:54592 (fd=5)
[Thread #2] Session started: 172.20.10.10:54592, fd=5
[Session] Reply: 220 Hybrid FTP service ready.
[Session] Command line: user testuser 3
[Session] Command: USER, Argument: testuser 3
[Session] Reply: 331 Username OK, need password.
[Session] Command line: pass 123
[Session] Command: PASS, Argument: 123
[Session] Reply: 230 Login successful.
[Session] Command line: user testuser
[Session] Command: USER, Argument: testuser
[Session] Reply: 331 Username OK, need password.
[Session] Command line: type i
[Session] Command: TYPE, Argument: i
[Session] Reply: 200 Type set to I (Binary).
[Session] Command line: pass testuser
[Session] Command: PASS, Argument: testuser
[Session] Reply: 230 Login successful.
[Session] Command line: pwd
[Session] Command: PWD, Argument:
[Session] Reply: 257 "/" is the current directory.
[Server] Thread #3 assigned to client 172.20.10.6:47292 (fd=6)
[Thread #3] Session started: 172.20.10.6:47292, fd=6
[Session] Reply: 220 Hybrid FTP service ready.
```

Hình 24: Log Server ghi nhận ba session, ba thread và các lệnh độc lập.

7.7 Ca kiểm thử 5: Multiple concurrent clients

7.7.1 Mục đích

- Server xử lý ít nhất ba client download đồng thời.
- Mỗi thread hoạt động độc lập, không làm sai dữ liệu của session khác.
- RDT vận hành ổn định với nhiều phiên đang truyền cùng lúc.

7.7.2 Các bước thực hiện

Trên Server, chuẩn bị ba file có kích thước khác nhau và bật log:

Listing 18: Chuẩn bị dữ liệu cho kiểm thử đồng thời.

```
1 mkdir -p data_test evidence
2 python3 -c "import os; open('data_test/file_A.bin', 'wb').write(os.urandom(512*1024))"
```

```

3 python3 -c "import os; open('data_test/file_B.bin','wb').write(os.urandom(256*1024))"
4 python3 -c "import os; open('data_test/file_C.bin','wb').write(os.urandom(128*1024))"
5 ls -lh data_test/file_*.bin
6 ./build/ftp_server 2121 2>&1 | tee evidence/server_concurrent_test.log

```

Trên Client, chạy ba tiến trình nền. Mỗi tiến trình đăng nhập, vào Passive Mode và download một file riêng:

Listing 19: Ba Client download đồng thời.

```

1 mkdir -p data_test/concurrent evidence
2 cd data_test/concurrent
3
4 (../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
5 user testuser
6 pass testpass
7 type I
8 pasv
9 retr file_A.bin
10 quit
11 COMMANDS
12 ) > ../../evidence/client1.log 2>&1 &
13
14 (../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
15 user testuser
16 pass testpass
17 type I
18 pasv
19 retr file_B.bin
20 quit
21 COMMANDS
22 ) > ../../evidence/client2.log 2>&1 &
23
24 (../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
25 user testuser
26 pass testpass
27 type I
28 pasv
29 retr file_C.bin
30 quit
31 COMMANDS
32 ) > ../../evidence/client3.log 2>&1 &
33
34 wait
35 cd ../../
36
37 for i in 1 2 3; do
38     if grep -q '226 Transfer complete' evidence/client${i}.log; then
39         echo "Client $i: SUCCESS"
40     else
41         echo "Client $i: FAILED"
42     fi
43 done

```

Kết quả đạt khi Server ghi nhận ba connection và ba thread có thời gian xử lý chồng lấn; cả ba Client đều nhận 226, đúng file và đúng kích thước.

7.7.3 Bảng chứng thu được

```

[SW-RECEIVER] Nhan seq=503 (1024 B) | tong=516096 B
[SW-RECEIVER] Gui ACK(503) | advertisedWin=8
[SW-RECEIVER] Nhan seq=504 (1024 B) | tong=517120 B
[SW-RECEIVER] Gui ACK(504) | advertisedWin=8
[SW-RECEIVER] Nhan seq=505 (1024 B) | tong=518144 B
[SW-RECEIVER] Gui ACK(505) | advertisedWin=8
[SW-RECEIVER] Nhan seq=506 (1024 B) | tong=519168 B
[SW-RECEIVER] Gui ACK(506) | advertisedWin=8
[SW-RECEIVER] Nhan seq=507 (1024 B) | tong=520192 B
[SW-RECEIVER] Gui ACK(507) | advertisedWin=8
[SW-RECEIVER] Nhan seq=508 (1024 B) | tong=521216 B
[SW-RECEIVER] Gui ACK(508) | advertisedWin=8
[SW-RECEIVER] Nhan seq=509 (1024 B) | tong=522240 B
[SW-RECEIVER] Gui ACK(509) | advertisedWin=8
[SW-RECEIVER] Nhan seq=510 (1024 B) | tong=523264 B
[SW-RECEIVER] Gui ACK(510) | advertisedWin=8
[SW-RECEIVER] Nhan seq=511 (1024 B) | tong=524288 B
[SW-RECEIVER] Gui ACK(511) | advertisedWin=8
[SW-RECEIVER] Nhan FIN -> Truyền hoàn tất, 524288 bytes.
[CLIENT] SHA-256 sau RETR: 58799235d0f4d3106a98acc090f22c286cc4463af24a6792dc28b4b8c0b91bbe (524288 bytes)
150 Opening data connection for file download.
226 Transfer complete.
  
```

Hình 25: Log Server cho thấy nhiều phiên RDT hoạt động đồng thời.

```

[SW-RECEIVER] Nhan seq=249 (1024 B) | tong=256000 B
[SW-RECEIVER] Gui ACK(249) | advertisedWin=8
[SW-RECEIVER] Nhan seq=250 (1024 B) | tong=257024 B
[SW-RECEIVER] Gui ACK(250) | advertisedWin=8
[SW-RECEIVER] Nhan seq=251 (1024 B) | tong=258048 B
[SW-RECEIVER] Gui ACK(251) | advertisedWin=8
[SW-RECEIVER] Nhan seq=252 (1024 B) | tong=259072 B
[SW-RECEIVER] Gui ACK(252) | advertisedWin=8
[SW-RECEIVER] Nhan seq=253 (1024 B) | tong=260096 B
[SW-RECEIVER] Gui ACK(253) | advertisedWin=8
[SW-RECEIVER] Nhan seq=254 (1024 B) | tong=261120 B
[SW-RECEIVER] Gui ACK(254) | advertisedWin=8
[SW-RECEIVER] Nhan seq=255 (1024 B) | tong=262144 B
[SW-RECEIVER] Gui ACK(255) | advertisedWin=8
[SW-RECEIVER] Nhan FIN -> Truyền hoàn tất, 262144 bytes.
[CLIENT] SHA-256 sau RETR: 26a72d5e4bc5b25e376e065ec0db184561de3aa514c4caab558cc66f5bd70d15 (262144 bytes)
150 Opening data connection for file download.
226 Transfer complete.
  
```

Hình 26: Một Client hoàn tất download file trong đợt kiểm thử đồng thời.

```

thhuule@LAPTOP-83RK0RML:/mnt/c/Users/ACER/FTP_PROJECT/data_test$ ../build/ftp_client 172.20.10.3 2121
[SW-RECEIVER] Nhan seq=121 (1024 B) | tong=124928 B
[SW-RECEIVER] Gui ACK(121) | advertisedWin=8
[SW-RECEIVER] Nhan seq=122 (1024 B) | tong=125952 B
[SW-RECEIVER] Gui ACK(122) | advertisedWin=8
[SW-RECEIVER] Nhan seq=123 (1024 B) | tong=126976 B
[SW-RECEIVER] Gui ACK(123) | advertisedWin=8
[SW-RECEIVER] Nhan seq=124 (1024 B) | tong=128000 B
[SW-RECEIVER] Gui ACK(124) | advertisedWin=8
[SW-RECEIVER] Nhan seq=125 (1024 B) | tong=129024 B
[SW-RECEIVER] Gui ACK(125) | advertisedWin=8
[SW-RECEIVER] Nhan seq=126 (1024 B) | tong=130048 B
[SW-RECEIVER] Gui ACK(126) | advertisedWin=8
[SW-RECEIVER] Nhan seq=127 (1024 B) | tong=131072 B
[SW-RECEIVER] Gui ACK(127) | advertisedWin=8
[SW-RECEIVER] Nhan FIN -> Truyen hoan tat, 131072 bytes.
[CLIENT] SHA-256 sau RETR: 06c210f0814d58f273e08c4725b2e7ed58107cc3652e82b70c084fbd0713ce6f (131072 bytes)
150 Opening data connection for file download.
226 Transfer complete.

```

Hình 27: Client khác hoàn tất phiên download độc lập.

```

PROBLEMS  OUTPUT  TERMINAL  ...
bash - Hybrid-FTP  +  -  [ ]  [ ]  ...  |  [ ]

● zoto1810@MSI:~/project/Hybrid-FTP$ echo "=== DOWNLOAD RESULTS ==="
for i in 1 2 3; do
  if grep -q "226 Transfer complete" session_tracking.log; then
    echo "✅ Client $i: SUCCESS"
  else
    echo "❌ Client $i: FAILED"
  fi
done
=== DOWNLOAD RESULTS ===
✅ Client 1: SUCCESS
✅ Client 2: SUCCESS
✅ Client 3: SUCCESS
○ zoto1810@MSI:~/project/Hybrid-FTP$

```

Hình 28: Script tổng hợp xác nhận cả ba Client đều thành công.

7.8 Ca kiểm thử 6: Transfer file ASCII Mode (TYPE A)

7.8.1 Mục đích

- Chứng minh Server chấp nhận TYPEA.
- Truyền file text thành công qua RDT.
- Nội dung file vẫn đọc được và khớp sau transfer.

7.8.2 Các bước thực hiện

Trên Client, tạo file text, tính hash rồi upload ở chế độ ASCII:

Listing 20: Upload file ở ASCII Mode.

```
1 mkdir -p data_test/ascii
```



```

2 cat > data_test/ascii/readme.txt <<'EOF'
3 Hybrid FTP System
4 Line 2: Testing ASCII Mode Transfer
5 Line 3: Special characters are preserved by the implementation
6 Line 4: Numbers: 1234567890
7 Line 5: Symbols are included in this test
8 EOF
9
10 sha256sum data_test/ascii/readme.txt
11 cd data_test/ascii
12 ../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
13 user testuser
14 pass testpass
15 type A
16 pasv
17 stor readme.txt
18 hash readme.txt
19 quit
20 COMMANDS
21 cd ../../

```

Trên Server, dùng cat, cat -A và sha256sum để kiểm tra nội dung, line ending và mã băm của file nhận được. Reply kỳ vọng gồm 200 cho TYPEA, 150 khi mở data channel, 226 khi hoàn tất và 213 cho lệnh HASH.

7.8.3 Bảng chứng thu được

```

ftp> type A
200 Type set to A (ASCII).

```

Hình 29: Client nhận reply 200 khi chọn TYPE A.

```

thhuule@LAPTOP-8JRK0RML:/mnt/c/Users/ACER/FTP_PROJECT/data_test$ ../../build/ftp_client 172.20.10.3 2121
[CLIENT] SHA-256 trước STOR: eeb67d95a7998d366d71aac4ed7a4eff2dd4b307a76c5edaceeadcb43955a25b (161 bytes)
[CLIENT] Gui file 'readme.txt' -> 'readme.txt' đến 172.20.10.3:46475
[RDT-SENDER] sendBuffer: 161 bytes
[SW-SENDER] Bắt đầu gửi 1 segment (MSS=1024, cwnd=1.0, ssthresh=8.0)
[SW-SENDER] Gui seq=0 (161 B) | cwnd=1.00 ssthresh=8.00 wnd=1
[SW-SENDER] Nhận ACK(0) | rcvwin=8 | cwnd=1.00 ssthresh=8.00
[SW-SENDER] Tất cả 1 segment đã ACK. cwnd_final=2.00, timeouts=0
[SW-SENDER] Gui FIN (lần 1)
[SW-SENDER] Nhận FINACK -> Truyền hoàn tất.
[CLIENT] Transfer OK: 161 bytes | cwnd=2.00 | timeouts=0 | segs=1
150 Opening data connection for file upload.
226 Transfer complete.

```

Hình 30: Log upload file text và reply 226 Transfer complete.

```
zoto1810@SI:~/project/Hybrid-FTP$ cat /home/zoto1810/project/Hybrid-FTP/server_storage/readme.txt
Hybrid FTP System
Line 2: Testing ASCII Mode Transfer
Line 3: Special chars: café, naïve, 中文
Line 4: Numbers: 1234567890
Line 5: Symbols: !@#%&*()
zoto1810@SI:~/project/Hybrid-FTP$ cat -A /home/zoto1810/project/Hybrid-FTP/server_storage/readme.txt
$
Hybrid FTP System$
$
Line 2: Testing ASCII Mode Transfer$
$
Line 3: Special chars: cafM-CM-), naM-CM-/ve, M-dM-8M- -M-fM-^VM-^G$
$
Line 4: Numbers: 1234567890$
$
Line 5: Symbols: !@#%&*())$
$
zoto1810@SI:~/project/Hybrid-FTP$ sha256sum /home/zoto1810/project/Hybrid-FTP/server_storage/readme.txt
eeb67d95a7998d366d71aac4ed7a4eff2dd4b307a76c5edaceeadcb43955a25b  /home/zoto1810/project/Hybrid-FTP/server_storage/readme.txt
zoto1810@SI:~/project/Hybrid-FTP$
```

Hình 31: Nội dung file text và SHA-256 được kiểm tra trên Server.

7.9 Ca kiểm thử 7: Transfer file Binary Mode (TYPE I)

7.9.1 Mục đích

- Chứng minh Server chấp nhận TYPE I.
- Không thực hiện chuyển đổi line ending.
- Dữ liệu binary, kể cả null byte, được truyền chính xác từng byte.

7.9.2 Các bước thực hiện

Trên Client, tạo file binary chứa bốn null byte, tính hash và xem hex dump:

Listing 21: Tạo dữ liệu binary chứa null byte.

```
1 mkdir -p data_test/binary
2 python3 -c "import os; d=os.urandom(10240); d=d[:1000]+b'\x00\x00\x00\x00'+d[1000:]; open('
  data_test/binary/binary_test.bin', 'wb').write(d)"
3 sha256sum data_test/binary/binary_test.bin \
4 > data_test/binary/binary_test.sha256.original
5 od -t x1 data_test/binary/binary_test.bin | head -5
```

Upload file ở Binary Mode, sau đó kiểm tra hash và hex dump trên Server:

Listing 22: Upload và xác nhận file Binary Mode.

```
1 cd data_test/binary
2 ../../build/ftp_client 192.168.1.100 2121 <<'COMMANDS'
3 user testuser
4 pass testpass
5 type I
```

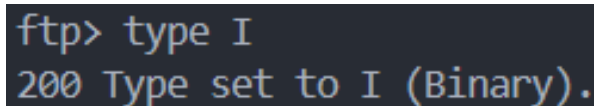
```

6 pasv
7 stor binary_test.bin
8 hash binary_test.bin
9 quit
10 COMMANDS
11 cd ../../
12
13 # Thuc hien tren Server
14 sha256sum data_test/binary_test.bin
15 od -t x1 data_test/binary_test.bin | head -5

```

Kết quả đạt khi Client nhận 200, 150, 226 và 213; SHA-256 ở hai máy giống nhau; hex dump trên Server vẫn có chuỗi 00000000 tại đúng vị trí.

7.9.3 Bảng chứng thu được

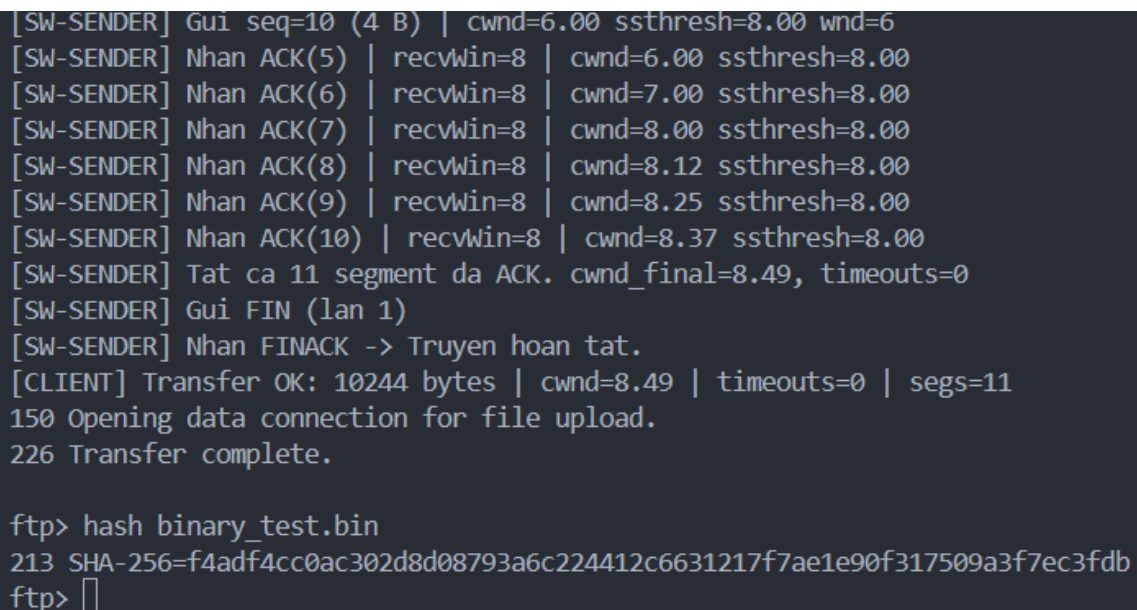


```

ftp> type I
200 Type set to I (Binary).

```

Hình 32: Client nhận reply 200 khi chọn TYPE I.



```

[SW-SENDER] Gui seq=10 (4 B) | cwnd=6.00 ssthresh=8.00 wnd=6
[SW-SENDER] Nhan ACK(5) | recvWin=8 | cwnd=6.00 ssthresh=8.00
[SW-SENDER] Nhan ACK(6) | recvWin=8 | cwnd=7.00 ssthresh=8.00
[SW-SENDER] Nhan ACK(7) | recvWin=8 | cwnd=8.00 ssthresh=8.00
[SW-SENDER] Nhan ACK(8) | recvWin=8 | cwnd=8.12 ssthresh=8.00
[SW-SENDER] Nhan ACK(9) | recvWin=8 | cwnd=8.25 ssthresh=8.00
[SW-SENDER] Nhan ACK(10) | recvWin=8 | cwnd=8.37 ssthresh=8.00
[SW-SENDER] Tat ca 11 segment da ACK. cwnd_final=8.49, timeouts=0
[SW-SENDER] Gui FIN (lan 1)
[SW-SENDER] Nhan FINACK -> Truyen hoan tat.
[CLIENT] Transfer OK: 10244 bytes | cwnd=8.49 | timeouts=0 | segs=11
150 Opening data connection for file upload.
226 Transfer complete.

ftp> hash binary_test.bin
213 SHA-256=f4adf4cc0ac302d8d08793a6c224412c6631217f7ae1e90f317509a3f7ec3fdb
ftp> 

```

Hình 33: Client upload file binary và nhận SHA-256 từ Server.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1
zoto1810@MSI:~/project/Hybrid-FTP/server_storage$ sha256sum binary_test.bin
f4adf4cc0ac302d8d08793a6c224412c6631217f7ae1e90f317509a3f7ec3fdb binary_test.bin
zoto1810@MSI:~/project/Hybrid-FTP/server_storage$ od -t x1 binary_test.bin | head -5
00000000 50 c7 37 4a ac 8f ea db 07 c2 56 87 ab 00 23 e5
00000020 36 a2 da 1f d6 10 ad 59 46 34 3c ea f1 98 85 52
00000040 61 99 6b 3c 48 b4 71 65 d2 36 63 2e b8 f1 31 c5
00000060 04 19 24 54 42 90 4f ba e4 29 e7 e3 6c 76 90 86
00000100 d8 92 cc e1 e7 c6 94 b6 1c d6 03 86 88 a1 9d 2e
zoto1810@MSI:~/project/Hybrid-FTP/server_storage$ 

```

Hình 34: Server xác nhận SHA-256 và null byte trong hex dump.

7.10 Tóm tắt bằng chứng

7.10.1 Bảng kiểm tra hoàn thành

Bảng 18: Tổng hợp evidence cho bảy ca kiểm thử.

#	Ca kiểm thử	Status	Log file	Ảnh bằng chứng
1	Upload STOR	✓ PASS	server_upload_test.log	Server/client upload terminal
2	Download RETR	✓ PASS	server_download_test.log	Server/client download terminal
3	SHA-256 Integrity	✓ PASS	hash_comparison.txt	Hash trước/sau transfer
4	Session Management	✓ PASS	session_tracking.log	Session và thread table
5	Concurrent Clients	✓ PASS	server_concurrent_test.log	Ba phiên đồng thời
6	ASCII Mode	✓ PASS	server_ascii_test.log	TYPE A và nội dung file
7	Binary Mode	✓ PASS	server_binary_test.log	TYPE I, hash và hex dump

7.10.2 Lưu giữ evidence files

Sau mỗi đợt chạy, sao chép log, hash và ảnh chụp vào thư mục evidence/, sau đó đóng gói để nộp kèm báo cáo:

Listing 23: Đóng gói evidence trên mỗi máy.

```

1 mkdir -p evidence/archive
2 cp evidence/server_*.log evidence/archive/ || true
3 cp evidence/session_tracking.log evidence/archive/ || true
4 cp evidence/client*.log evidence/archive/ || true
5 cp data_test/binary/binary_test.sha256.original \
6   evidence/archive/ || true
7 tar -czf evidence/ftp_evidence.tar.gz evidence/archive

```

```
8 ls -lh evidence/ftp_evidence.tar.gz
```

7.11 Dọn dẹp sau khi test

Chỉ xóa dữ liệu thử sau khi đã đóng gói và sao lưu evidence cần nộp:

Listing 24: Dọn dữ liệu test trên Server và Client.

```
1 pkill -f ftp_server
2 rm -rf data_test
3 rm -f evidence/server_*.log evidence/session_tracking.log
4 rm -f evidence/client*.log
```

Kết luận nghiệm thu

Các ca kiểm thử và ảnh chụp trong chương này cung cấp bằng chứng cho hoạt động của Hybrid FTP trên hai máy tính: truyền tin cậy STOR/RETR, quản lý session đa luồng, nhiều Client đồng thời, chế độ ASCII/Binary và xác minh tính toàn vẹn bằng SHA-256.